Problem Statement

This tutorial takes a practical and coding-focused approach. We'll learn how to apply *logistic regression* to a real-world dataset from Kaggle:

> **QUESTION**: The Rain in Australia dataset contains about 10 years of daily weather observations from numerous Australian weather stations. Here's a small sample from the dataset:
>
> | Date | Location | MinTemp | MaxTemp | Rainfall | Evaporation | Sunshine | Cloud3pm | Temp9am | Temp3pm | RainToday | RainTomorrow |
> |---|---|---|---|---|---|---|---|---|---|---|---|
> | 2008-09-21 | Melbourne | 6.5 | 19.8 | 0.4 | 4.2 | 10.6 | 3.0 | 13.0 | 19.4 | No | No |
> | 2009-07-06 | Sale | 4.9 | 13.0 | 0.0 | 2.0 | 6.8 | 6.0 | 8.6 | 11.7 | No | No |
> | 2010-11-20 | GoldCoast | 18.8 | 26.4 | 2.0 | NaN | NaN | NaN | 24.0 | 22.1 | Yes | No |
> | 2010-11-22 | PearceRAAF | 19.4 | 27.4 | 1.8 | NaN | 10.7 | 3.0 | 24.4 | 25.8 | Yes | No |
> | 2012-04-26 | Nuriootpa | 5.1 | 16.6 | 0.0 | 1.4 | 1.4 | 7.0 | 12.1 | 15.7 | No | No |
> | 2013-07-06 | Sydney | 7.8 | 17.4 | 0.0 | 4.2 | 9.8 | 0.0 | 10.2 | 17.1 | No | No |
> | 2014-04-22 | Perth | 7.7 | 23.7 | 0.0 | 4.0 | 10.5 | 1.0 | 16.7 | 21.8 | No | No |
> | 2014-06-08 | Wollongong | 11.1 | 16.8 | 0.0 | NaN | NaN | 1.0 | 14.0 | 15.9 | No | No |
> | 2016-04-13 | Sale | 10.8 | 19.0 | 0.0 | NaN | NaN | 1.0 | 16.1 | 18.1 | No | No |
> | 2017-04-11 | Albany | 13.0 | NaN | 0.0 | 4.0 | NaN | NaN | 17.8 | NaN | No | NaN |
>
> As a data scientist at the Bureau of Meteorology, you are tasked with creating a fully-automated system that can use today's weather data for a given location to predict whether it will rain at the location tomorrow.
>
> 
>

Logistic Regression for Solving Classification Problems

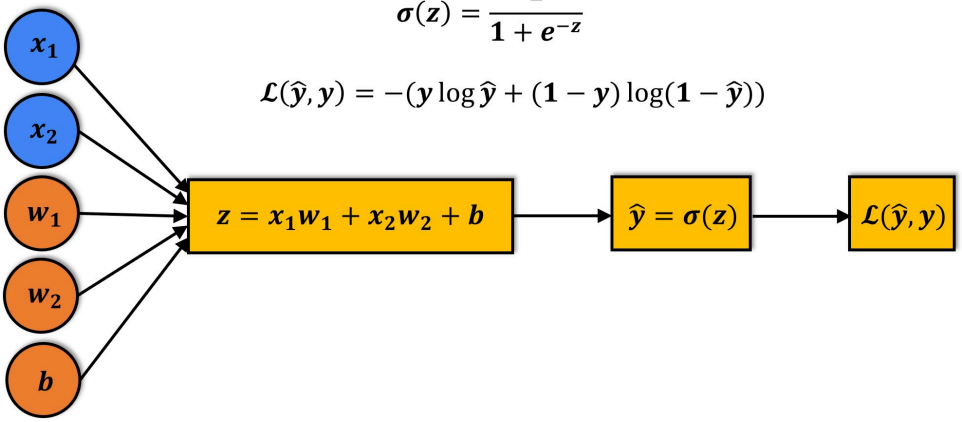
Logistic regression is a commonly used technique for solving binary classification problems. In a logistic regression model:

- we take linear combination (or weighted sum of the input features)
- we apply the sigmoid function to the result to obtain a number between 0 and 1
- this number represents the probability of the input being classified as "Yes"
- instead of RMSE, the cross entropy loss function is used to evaluate the results

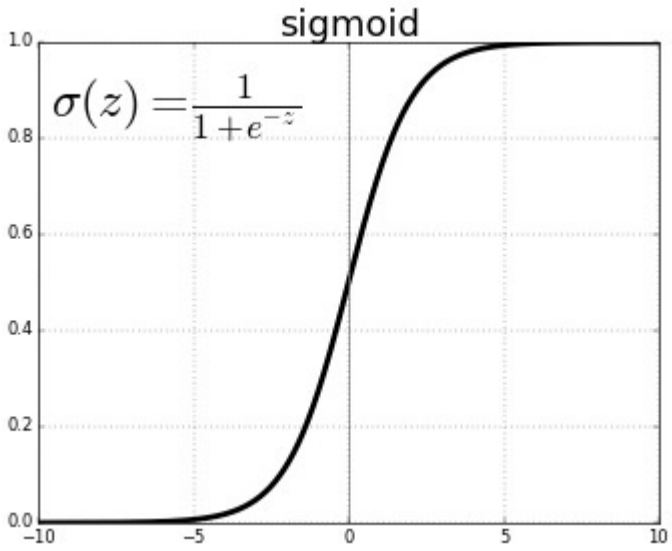
Here's a visual summary of how a logistic regression model is structured (source):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$



The sigmoid function applied to the linear combination of inputs has the following formula:



The output of the sigmoid function is called a logistic, hence the name *logistic regression*. For a mathematical discussion of logistic regression, sigmoid activation and cross entropy, check out this YouTube playlist. Logistic regression can also be applied to multi-class classification problems, with a few modifications.

In [48]:
```
!pip install opendatasets --upgrade --quiet
```

In [49]:
```
%pip install -U opendatasets
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: opendatasets in c:\users\prachita\appdata\roaming
\python\python313\site-packages (0.1.22)
Requirement already satisfied: tqdm in c:\programdata\anaconda3\lib\site-packages
(from opendatasets) (4.67.1)
Requirement already satisfied: kaggle in c:\users\prachita\appdata\roaming\python
\python313\site-packages (from opendatasets) (2.2.4)
Requirement already satisfied: click in c:\programdata\anaconda3\lib\site-package
s (from opendatasets) (8.2.1)
Requirement already satisfied: colorama in c:\programdata\anaconda3\lib\site-pack
ages (from click->opendatasets) (0.4.6)
Requirement already satisfied: bleach in c:\programdata\anaconda3\lib\site-packag
es (from kaggle->opendatasets) (6.3.0)
Requirement already satisfied: jupytext in c:\users\prachita\appdata\roaming\pyth
on\python313\site-packages (from kaggle->opendatasets) (1.19.5)
Requirement already satisfied: kagglesdk<1.0,>=0.1.35 in c:\users\prachita\appdat
a\roaming\python\python313\site-packages (from kaggle->opendatasets) (0.1.37)
Requirement already satisfied: packaging in c:\programdata\anaconda3\lib\site-pac
kages (from kaggle->opendatasets) (25.0)
Requirement already satisfied: protobuf in c:\programdata\anaconda3\lib\site-pack
ages (from kaggle->opendatasets) (5.29.3)
Requirement already satisfied: python-dateutil in c:\programdata\anaconda3\lib\si
te-packages (from kaggle->opendatasets) (2.9.0.post0)
Requirement already satisfied: python-dotenv in c:\programdata\anaconda3\lib\site
-packages (from kaggle->opendatasets) (1.1.0)
Requirement already satisfied: python-slugify in c:\programdata\anaconda3\lib\sit
e-packages (from kaggle->opendatasets) (8.0.4)
Requirement already satisfied: requests in c:\programdata\anaconda3\lib\site-pack
ages (from kaggle->opendatasets) (2.32.5)
Requirement already satisfied: urllib3>=1.15.1 in c:\programdata\anaconda3\lib\si
te-packages (from kaggle->opendatasets) (2.5.0)
Requirement already satisfied: webencodings in c:\programdata\anaconda3\lib\site-
packages (from bleach->kaggle->opendatasets) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in c:\programdata\anaconda3\li
b\site-packages (from jupytext->kaggle->opendatasets) (2.2.0)
Requirement already satisfied: mdit-py-plugins in c:\programdata\anaconda3\lib\si
te-packages (from jupytext->kaggle->opendatasets) (0.5.0)
Requirement already satisfied: nbformat in c:\programdata\anaconda3\lib\site-pack
ages (from jupytext->kaggle->opendatasets) (5.10.4)
Requirement already satisfied: pyyaml in c:\programdata\anaconda3\lib\site-packag
es (from jupytext->kaggle->opendatasets) (6.0.3)
Requirement already satisfied: mdurl~=0.1 in c:\programdata\anaconda3\lib\site-pa
ckages (from markdown-it-py>=1.0->jupytext->kaggle->opendatasets) (0.1.2)
Requirement already satisfied: fastjsonschema>=2.15 in c:\programdata\anaconda3\l
ib\site-packages (from nbformat->jupytext->kaggle->opendatasets) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in c:\programdata\anaconda3\lib\si
te-packages (from nbformat->jupytext->kaggle->opendatasets) (4.25.0)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in c:\programdata\anaco
nda3\lib\site-packages (from nbformat->jupytext->kaggle->opendatasets) (5.8.1)
Requirement already satisfied: traitlets>=5.1 in c:\programdata\anaconda3\lib\sit
e-packages (from nbformat->jupytext->kaggle->opendatasets) (5.14.3)
Requirement already satisfied: attrs>=22.2.0 in c:\programdata\anaconda3\lib\site
-packages (from jsonschema>=2.6->nbformat->jupytext->kaggle->opendatasets) (25.4.
0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\program
data\anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat->jupytext->kaggl
e->opendatasets) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in c:\programdata\anaconda3\li
b\site-packages (from jsonschema>=2.6->nbformat->jupytext->kaggle->opendatasets)
(0.37.0)
```

```
Requirement already satisfied: rpds-py>=0.7.1 in c:\programdata\anaconda3\lib\sit
e-packages (from jsonschema>=2.6->nbformat->jupytext->kaggle->opendatasets) (0.2
8.0)
Requirement already satisfied: platformdirs>=2.5 in c:\programdata\anaconda3\lib
\site-packages (from jupyter-core!=5.0.*,>=4.12->nbformat->jupytext->kaggle->open
datasets) (4.5.0)
Requirement already satisfied: pywin32>=300 in c:\programdata\anaconda3\lib\site-
packages (from jupyter-core!=5.0.*,>=4.12->nbformat->jupytext->kaggle->opendatase
ts) (311)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-pack
ages (from python-dateutil->kaggle->opendatasets) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in c:\programdata\anaconda3\li
b\site-packages (from python-slugify->kaggle->opendatasets) (1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\programdata\anacond
a3\lib\site-packages (from requests->kaggle->opendatasets) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\programdata\anaconda3\lib\site-
packages (from requests->kaggle->opendatasets) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in c:\programdata\anaconda3\lib
\site-packages (from requests->kaggle->opendatasets) (2026.4.22)
Note: you may need to restart the kernel to use updated packages.
```

In [50]:
```
import sys
print(sys.executable)
```

```
c:\ProgramData\anaconda3\python.exe
```

In [51]:
```
import sys
!{sys.executable} -m pip install -U opendatasets
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: opendatasets in c:\users\prachita\appdata\roaming
\python\python313\site-packages (0.1.22)
Requirement already satisfied: tqdm in c:\programdata\anaconda3\lib\site-packages
(from opendatasets) (4.67.1)
Requirement already satisfied: kaggle in c:\users\prachita\appdata\roaming\python
\python313\site-packages (from opendatasets) (2.2.4)
Requirement already satisfied: click in c:\programdata\anaconda3\lib\site-package
s (from opendatasets) (8.2.1)
Requirement already satisfied: colorama in c:\programdata\anaconda3\lib\site-pack
ages (from click->opendatasets) (0.4.6)
Requirement already satisfied: bleach in c:\programdata\anaconda3\lib\site-packag
es (from kaggle->opendatasets) (6.3.0)
Requirement already satisfied: jupytext in c:\users\prachita\appdata\roaming\pyth
on\python313\site-packages (from kaggle->opendatasets) (1.19.5)
Requirement already satisfied: kagglesdk<1.0,>=0.1.35 in c:\users\prachita\appdat
a\roaming\python\python313\site-packages (from kaggle->opendatasets) (0.1.37)
Requirement already satisfied: packaging in c:\programdata\anaconda3\lib\site-pac
kages (from kaggle->opendatasets) (25.0)
Requirement already satisfied: protobuf in c:\programdata\anaconda3\lib\site-pack
ages (from kaggle->opendatasets) (5.29.3)
Requirement already satisfied: python-dateutil in c:\programdata\anaconda3\lib\si
te-packages (from kaggle->opendatasets) (2.9.0.post0)
Requirement already satisfied: python-dotenv in c:\programdata\anaconda3\lib\site
-packages (from kaggle->opendatasets) (1.1.0)
Requirement already satisfied: python-slugify in c:\programdata\anaconda3\lib\sit
e-packages (from kaggle->opendatasets) (8.0.4)
Requirement already satisfied: requests in c:\programdata\anaconda3\lib\site-pack
ages (from kaggle->opendatasets) (2.32.5)
Requirement already satisfied: urllib3>=1.15.1 in c:\programdata\anaconda3\lib\si
te-packages (from kaggle->opendatasets) (2.5.0)
Requirement already satisfied: webencodings in c:\programdata\anaconda3\lib\site-
packages (from bleach->kaggle->opendatasets) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in c:\programdata\anaconda3\li
b\site-packages (from jupytext->kaggle->opendatasets) (2.2.0)
Requirement already satisfied: mdit-py-plugins in c:\programdata\anaconda3\lib\si
te-packages (from jupytext->kaggle->opendatasets) (0.5.0)
Requirement already satisfied: nbformat in c:\programdata\anaconda3\lib\site-pack
ages (from jupytext->kaggle->opendatasets) (5.10.4)
Requirement already satisfied: pyyaml in c:\programdata\anaconda3\lib\site-packag
es (from jupytext->kaggle->opendatasets) (6.0.3)
Requirement already satisfied: mdurl~=0.1 in c:\programdata\anaconda3\lib\site-pa
ckages (from markdown-it-py>=1.0->jupytext->kaggle->opendatasets) (0.1.2)
Requirement already satisfied: fastjsonschema>=2.15 in c:\programdata\anaconda3\l
ib\site-packages (from nbformat->jupytext->kaggle->opendatasets) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in c:\programdata\anaconda3\lib\si
te-packages (from nbformat->jupytext->kaggle->opendatasets) (4.25.0)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in c:\programdata\anaco
nda3\lib\site-packages (from nbformat->jupytext->kaggle->opendatasets) (5.8.1)
Requirement already satisfied: traitlets>=5.1 in c:\programdata\anaconda3\lib\sit
e-packages (from nbformat->jupytext->kaggle->opendatasets) (5.14.3)
Requirement already satisfied: attrs>=22.2.0 in c:\programdata\anaconda3\lib\site
-packages (from jsonschema>=2.6->nbformat->jupytext->kaggle->opendatasets) (25.4.
0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\program
data\anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat->jupytext->kaggl
e->opendatasets) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in c:\programdata\anaconda3\li
b\site-packages (from jsonschema>=2.6->nbformat->jupytext->kaggle->opendatasets)
(0.37.0)
```

```
Requirement already satisfied: rpds-py>=0.7.1 in c:\programdata\anaconda3\lib\sit
e-packages (from jsonschema>=2.6->nbformat->jupytext->kaggle->opendatasets) (0.2
8.0)
Requirement already satisfied: platformdirs>=2.5 in c:\programdata\anaconda3\lib
\site-packages (from jupyter-core!=5.0.*,>=4.12->nbformat->jupytext->kaggle->open
datasets) (4.5.0)
Requirement already satisfied: pywin32>=300 in c:\programdata\anaconda3\lib\site-
packages (from jupyter-core!=5.0.*,>=4.12->nbformat->jupytext->kaggle->opendatase
ts) (311)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-pack
ages (from python-dateutil->kaggle->opendatasets) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in c:\programdata\anaconda3\li
b\site-packages (from python-slugify->kaggle->opendatasets) (1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\programdata\anacond
a3\lib\site-packages (from requests->kaggle->opendatasets) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\programdata\anaconda3\lib\site-
packages (from requests->kaggle->opendatasets) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in c:\programdata\anaconda3\lib
\site-packages (from requests->kaggle->opendatasets) (2026.4.22)
```

In [52]:
```
%pip install legacy-cgi
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: legacy-cgi in c:\users\prachita\appdata\roaming\py
thon\python313\site-packages (2.6.4)
Note: you may need to restart the kernel to use updated packages.
```

In [53]:
```
import opendatasets as od
print("Working!")
```

```
Working!
```

In [54]:
```
od.version()
```

Out[54]:
```
'0.1.22'
```

In [55]:
```
dataset_url = 'https://www.kaggle.com/jsphyg/weather-dataset-rattle-package'
```

In [56]:
```
import pandas as pd
```

In [57]:
```
raw_df = pd.read_csv('weatherAUS.csv')
```

In [58]:
```
raw_df
```

Out[58]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindG
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	
...	...	...	...	...	...	...	...	
145455	2017-06-21	Uluru	2.8	23.4	0.0	NaN	NaN	
145456	2017-06-22	Uluru	3.6	25.3	0.0	NaN	NaN	
145457	2017-06-23	Uluru	5.4	26.9	0.0	NaN	NaN	
145458	2017-06-24	Uluru	7.8	27.0	0.0	NaN	NaN	
145459	2017-06-25	Uluru	14.9	NaN	0.0	NaN	NaN	

145460 rows × 23 columns

We have 23 columns which will give information about 145460 data

In [59]:
```
raw_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   Date           145460 non-null  object 
 1   Location       145460 non-null  object 
 2   MinTemp        143975 non-null  float64
 3   MaxTemp        144199 non-null  float64
 4   Rainfall       142199 non-null  float64
 5   Evaporation    82670 non-null   float64
 6   Sunshine       75625 non-null   float64
 7   WindGustDir    135134 non-null  object 
 8   WindGustSpeed  135197 non-null  float64
 9   WindDir9am     134894 non-null  object 
 10  WindDir3pm     141232 non-null  object 
 11  WindSpeed9am   143693 non-null  float64
 12  WindSpeed3pm   142398 non-null  float64
 13  Humidity9am    142806 non-null  float64
 14  Humidity3pm    140953 non-null  float64
 15  Pressure9am    130395 non-null  float64
 16  Pressure3pm    130432 non-null  float64
 17  Cloud9am       89572 non-null   float64
 18  Cloud3pm       86102 non-null   float64
 19  Temp9am        143693 non-null  float64
 20  Temp3pm        141851 non-null  float64
 21  RainToday      142199 non-null  object 
 22  RainTomorrow   142193 non-null  object 
dtypes: float64(16), object(7)
memory usage: 25.5+ MB
```

In [60]:
```
raw_df.dropna(subset=['RainToday','RainTomorrow'], inplace=True)
```

In [61]:
```
raw_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 140787 entries, 0 to 145458
Data columns (total 23 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   Date           140787 non-null  object 
 1   Location       140787 non-null  object 
 2   MinTemp        140319 non-null  float64
 3   MaxTemp        140480 non-null  float64
 4   Rainfall       140787 non-null  float64
 5   Evaporation    81093 non-null   float64
 6   Sunshine       73982 non-null   float64
 7   WindGustDir    131624 non-null  object 
 8   WindGustSpeed  131682 non-null  float64
 9   WindDir9am     131127 non-null  object 
 10  WindDir3pm     137117 non-null  object 
 11  WindSpeed9am   139732 non-null  float64
 12  WindSpeed3pm   138256 non-null  float64
 13  Humidity9am    139270 non-null  float64
 14  Humidity3pm    137286 non-null  float64
 15  Pressure9am    127044 non-null  float64
 16  Pressure3pm    127018 non-null  float64
 17  Cloud9am       88162 non-null   float64
 18  Cloud3pm       84693 non-null   float64
 19  Temp9am        140131 non-null  float64
 20  Temp3pm        138163 non-null  float64
 21  RainToday      140787 non-null  object 
 22  RainTomorrow   140787 non-null  object 
dtypes: float64(16), object(7)
memory usage: 25.8+ MB
```

Exploring and Analysing Data via Visualization

In [62]:
```
!pip install plotly matplotlib seaborn --quiet
```

In [63]:
```
import plotly.express as px
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

sns.set_style('darkgrid')
matplotlib.rcParams['font.size'] = 14
matplotlib.rcParams['figure.figsize'] = (10, 6)
matplotlib.rcParams['figure.facecolor'] = '#00000000'
```

In [64]:
```
raw_df.Location.unique()
```

Out[64]:
```
array(['Albury', 'BadgerysCreek', 'Cobar', 'CoffsHarbour', 'Moree',
       'Newcastle', 'NorahHead', 'NorfolkIsland', 'Penrith', 'Richmond',
       'Sydney', 'SydneyAirport', 'WaggaWagga', 'Williamtown',
       'Wollongong', 'Canberra', 'Tuggeranong', 'MountGinini', 'Ballarat',
       'Bendigo', 'Sale', 'MelbourneAirport', 'Melbourne', 'Mildura',
       'Nhil', 'Portland', 'Watsonia', 'Dartmoor', 'Brisbane', 'Cairns',
       'GoldCoast', 'Townsville', 'Adelaide', 'MountGambier', 'Nuriootpa',
       'Woomera', 'Albany', 'Witchcliffe', 'PearceRAAF', 'PerthAirport',
       'Perth', 'SalmonGums', 'Walpole', 'Hobart', 'Launceston',
       'AliceSprings', 'Darwin', 'Katherine', 'Uluru'], dtype=object)
```

In [65]:
```
raw_df.Location.nunique()
```

Out[65]: 49

In [66]:
```
 px.histogram(raw_df, x='Location', title='Location vs Rainy Days', color='RainT
```

In [67]:
```
px.histogram(raw_df, x='Temp3pm', title='Temperature at 3pm vs Rain Tomorrow', c
```

In [68]:
```
px.histogram(raw_df,
             x='RainTomorrow',
             color='RainToday',
             title='Rain Tomorrow vs. Rain Today')
```

In [69]:
```
px.scatter(raw_df.sample(3000), title='MinTemp vs MaxTemp', x='MinTemp', y='MaxT
```

In [70]:
```
px.scatter(raw_df.sample(2000),
           title='Temp (3 pm) vs. Humidity (3 pm)',
           x='Temp3pm',
           y='Humidity3pm',
           color='RainTomorrow')
```

Model

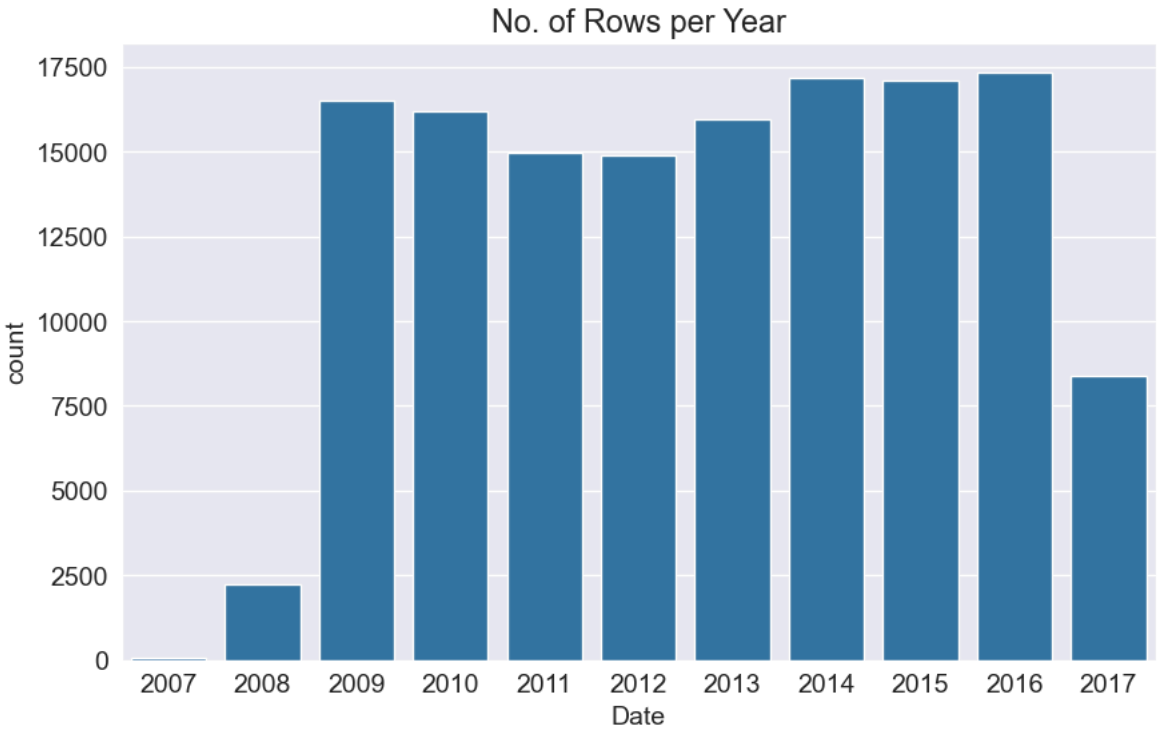
In [71]:
```
from sklearn.model_selection import train_test_split
```

In [72]:
```
train_val_df, test_df = train_test_split(raw_df, test_size=0.2, random_state=42)
train_df, val_df = train_test_split(train_val_df, test_size=0.20, random_state=4
```

In [73]:
```
print(f"Train shape: {train_df.shape}")
print(f"Validation shape: {val_df.shape}")
print(f"Test shape: {test_df.shape}")
```

```
Train shape: (90103, 23)
Validation shape: (22526, 23)
Test shape: (28158, 23)
```

In [74]:
```
plt.title('No. of Rows per Year')
sns.countplot(x=pd.to_datetime(raw_df.Date).dt.year);
```



In [75]:
```
year = pd.to_datetime(raw_df.Date).dt.year

train_df= raw_df[year<2015]
val_df= raw_df[year==2015]
test_df= raw_df[year>2015]
```

In [76]:
```
train_df
```

Out[76]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindG
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	
...	...	...	...	...	...	...	...	
144548	2014-12-27	Uluru	16.9	33.2	0.0	NaN	NaN	
144549	2014-12-28	Uluru	15.1	36.8	0.0	NaN	NaN	
144550	2014-12-29	Uluru	17.3	37.8	0.0	NaN	NaN	
144551	2014-12-30	Uluru	20.1	38.5	0.0	NaN	NaN	
144552	2014-12-31	Uluru	22.5	39.6	0.0	NaN	NaN	

97988 rows × 23 columns

In [77]:
```
val_df
```

Out[77]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindG
2133	2015-01-01	Albury	11.4	33.5	0.0	NaN	NaN	
2134	2015-01-02	Albury	15.5	39.6	0.0	NaN	NaN	
2135	2015-01-03	Albury	17.1	38.3	0.0	NaN	NaN	
2136	2015-01-04	Albury	26.0	33.1	0.0	NaN	NaN	
2137	2015-01-05	Albury	19.0	35.2	0.0	NaN	NaN	
...	...	...	...	...	...	...	...	
144913	2015-12-27	Uluru	20.5	34.7	0.0	NaN	NaN	
144914	2015-12-28	Uluru	18.0	36.4	0.0	NaN	NaN	
144915	2015-12-29	Uluru	17.5	37.1	0.0	NaN	NaN	
144916	2015-12-30	Uluru	20.0	38.9	0.0	NaN	NaN	
144917	2015-12-31	Uluru	19.3	37.4	0.0	NaN	NaN	

17089 rows × 23 columns

In [78]:
```
test_df
```

Out[78]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindG
2498	2016-01-01	Albury	20.4	37.6	0.0	NaN	NaN	
2499	2016-01-02	Albury	20.9	33.6	0.4	NaN	NaN	
2500	2016-01-03	Albury	18.4	23.1	2.2	NaN	NaN	
2501	2016-01-04	Albury	17.3	23.7	15.6	NaN	NaN	
2502	2016-01-05	Albury	15.5	22.9	6.8	NaN	NaN	
...	...	...	...	...	...	...	...	
145454	2017-06-20	Uluru	3.5	21.8	0.0	NaN	NaN	
145455	2017-06-21	Uluru	2.8	23.4	0.0	NaN	NaN	
145456	2017-06-22	Uluru	3.6	25.3	0.0	NaN	NaN	
145457	2017-06-23	Uluru	5.4	26.9	0.0	NaN	NaN	
145458	2017-06-24	Uluru	7.8	27.0	0.0	NaN	NaN	

25710 rows × 23 columns

In [79]:
```
input_cols = list(train_df.columns)[1:-1]
target_col = 'RainTomorrow'
```

In [80]:
```
print(input_cols)
```

```
['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation', 'Sunshine', 'WindGu
stDir', 'WindGustSpeed', 'WindDir9am', 'WindDir3pm', 'WindSpeed9am', 'WindSpeed3p
m', 'Humidity9am', 'Humidity3pm', 'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Clou
d3pm', 'Temp9am', 'Temp3pm', 'RainToday']
```

In [81]:
```
train_inputs = train_df[input_cols].copy()
train_targets = train_df[target_col].copy()
val_inputs = val_df[input_cols].copy()
val_targets = val_df[target_col].copy()
test_inputs = test_df[input_cols].copy()
test_targets = test_df[target_col].copy()
```

In [82]:
```
train_inputs
```

Out[82]:

	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir
0	Albury	13.4	22.9	0.6	NaN	NaN	W
1	Albury	7.4	25.1	0.0	NaN	NaN	WNW
2	Albury	12.9	25.7	0.0	NaN	NaN	WSW
3	Albury	9.2	28.0	0.0	NaN	NaN	NE
4	Albury	17.5	32.3	1.0	NaN	NaN	W
...	...	...	...	...	...	...	...
144548	Uluru	16.9	33.2	0.0	NaN	NaN	SSE
144549	Uluru	15.1	36.8	0.0	NaN	NaN	NE
144550	Uluru	17.3	37.8	0.0	NaN	NaN	ESE
144551	Uluru	20.1	38.5	0.0	NaN	NaN	ESE
144552	Uluru	22.5	39.6	0.0	NaN	NaN	WNW

97988 rows × 21 columns

In [83]:
```
train_targets
```

Out[83]:
```
0         No
1         No
2         No
3         No
4         No
          ..
144548    No
144549    No
144550    No
144551    No
144552    No
Name: RainTomorrow, Length: 97988, dtype: object
```

In [84]:
```
import numpy as np
```

In [85]:
```
numeric_cols = train_inputs.select_dtypes(include=np.number).columns.tolist()
categorical_cols = train_inputs.select_dtypes('object').columns.tolist()
```

In [86]:
```
train_inputs[numeric_cols].describe()
```

Out[86]:

	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGust
count	97674.000000	97801.000000	97988.000000	61657.000000	57942.000000	91160.
mean	12.007831	23.022202	2.372935	5.289991	7.609004	40.
std	6.347175	6.984397	8.518819	3.952010	3.788813	13.
min	-8.500000	-4.100000	0.000000	0.000000	0.000000	6.
25%	7.500000	17.900000	0.000000	2.600000	4.800000	31.
50%	11.800000	22.400000	0.000000	4.600000	8.500000	39.
75%	16.600000	27.900000	0.800000	7.200000	10.600000	48.
max	33.900000	48.100000	371.000000	82.400000	14.300000	135.

In [89]:
```
train_inputs[categorical_cols].nunique()
```

Out[89]:
```
Location       49
WindGustDir    16
WindDir9am     16
WindDir3pm     16
RainToday       2
dtype: int64
```

Imputing Missing Numeric Data

Machine learning models can't work with missing numerical data. The process of filling missing values is called imputation.

	col1	col2	col3	col4	col5
0	2	5.0	3.0	6	NaN
1	9	NaN	9.0	0	7.0
2	19	17.0	NaN	9	NaN

mean()

	col1	col2	col3	col4	col5
0	2.0	5.0	3.0	6.0	7.0
1	9.0	11.0	9.0	0.0	7.0
2	19.0	17.0	6.0	9.0	7.0

There are several techniques for imputation, but we'll use the most basic one: replacing missing values with the average value in the column using the `SimpleImputer` class from `sklearn.impute`.

In [90]:
```
from sklearn.impute import SimpleImputer
```

In [91]:
```
imputer = SimpleImputer(strategy='mean')
```

In [92]:
```
raw_df[numeric_cols].isna().sum()
```

Out[92]:
```
MinTemp           468
MaxTemp           307
Rainfall            0
Evaporation     59694
Sunshine        66805
WindGustSpeed    9105
WindSpeed9am     1055
WindSpeed3pm     2531
Humidity9am      1517
Humidity3pm      3501
Pressure9am     13743
Pressure3pm     13769
Cloud9am        52625
Cloud3pm        56094
Temp9am           656
Temp3pm          2624
dtype: int64
```

In [93]:
```
train_inputs[numeric_cols].isna().sum()
```

Out[93]:
```
MinTemp           314
MaxTemp           187
Rainfall            0
Evaporation     36331
Sunshine        40046
WindGustSpeed    6828
WindSpeed9am      874
WindSpeed3pm     1069
Humidity9am      1052
Humidity3pm      1116
Pressure9am      9112
Pressure3pm      9131
Cloud9am        34988
Cloud3pm        36022
Temp9am           574
Temp3pm           596
dtype: int64
```

In [94]:
```
imputer.fit(raw_df[numeric_cols])
```

Out[94]:

▾ SimpleImputer

▶ Parameters

In [95]:
```
list(imputer.statistics_)
```

Out[95]:
```
[np.float64(12.184823865620478),
 np.float64(23.235120301822324),
 np.float64(2.349974074310839),
 np.float64(5.472515506887154),
 np.float64(7.630539861047283),
 np.float64(39.97051988882308),
 np.float64(13.990496092519967),
 np.float64(18.631140782316862),
 np.float64(68.82683277087672),
 np.float64(51.44928834695453),
 np.float64(1017.6545771543716),
 np.float64(1015.2579625879797),
 np.float64(4.431160817585808),
 np.float64(4.499250233195188),
 np.float64(16.987066387879914),
 np.float64(21.693182690011074)]
```

In [96]:
```
train_inputs[numeric_cols] = imputer.transform(train_inputs[numeric_cols])
val_inputs[numeric_cols] = imputer.transform(val_inputs[numeric_cols])
test_inputs[numeric_cols] = imputer.transform(test_inputs[numeric_cols])
```

In [97]:
```
train_inputs[numeric_cols].isna().sum()
```

Out[97]:
```
MinTemp          0
MaxTemp          0
Rainfall         0
Evaporation      0
Sunshine         0
WindGustSpeed    0
WindSpeed9am     0
WindSpeed3pm     0
Humidity9am      0
Humidity3pm      0
Pressure9am      0
Pressure3pm      0
Cloud9am         0
Cloud3pm         0
Temp9am          0
Temp3pm          0
dtype: int64
```

Scaling Numeric Features

Another good practice is to scale numeric features to a small range of values e.g. $(0, 1)$ or $(-1, 1)$. Scaling numeric features ensures that no particular feature has a disproportionate impact on the model's loss. Optimization algorithms also work better in practice with smaller numbers.

The numeric columns in our dataset have varying ranges.

In [98]:
```
raw_df[numeric_cols].describe()
```

Out[98]:

	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindG
count	140319.000000	140480.00000	140787.000000	81093.000000	73982.000000	13168
mean	12.184824	23.23512	2.349974	5.472516	7.630540	3
std	6.403879	7.11450	8.465173	4.189132	3.781729	1
min	-8.500000	-4.80000	0.000000	0.000000	0.000000	
25%	7.600000	17.90000	0.000000	2.600000	4.900000	3
50%	12.000000	22.60000	0.000000	4.800000	8.500000	3
75%	16.800000	28.30000	0.800000	7.400000	10.700000	4
max	33.900000	48.10000	371.000000	145.000000	14.500000	13

In [99]:
```
from sklearn.preprocessing import MinMaxScaler
```

In [100…
```
?MinMaxScaler
```

```
Init signature: MinMaxScaler(feature_range=(0, 1), *, copy=True, clip=False)
Docstring:
Transform features by scaling each feature to a given range.

This estimator scales and translates each feature individually such
that it is in the given range on the training set, e.g. between
zero and one.

The transformation is given by::

    X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
    X_scaled = X_std * (max - min) + min

where min, max = feature_range.

This transformation is often used as an alternative to zero mean,
unit variance scaling.

`MinMaxScaler` doesn't reduce the effect of outliers, but it linearly
scales them down into a fixed range, where the largest occurring data point
corresponds to the maximum value and the smallest one corresponds to the
minimum value. For an example visualization, refer to :ref:`Compare
MinMaxScaler with other scalers <plot_all_scaling_minmax_scaler_section>`.

Read more in the :ref:`User Guide <preprocessing_scaler>`.

Parameters
----------
feature_range : tuple (min, max), default=(0, 1)
    Desired range of transformed data.

copy : bool, default=True
    Set to False to perform inplace row normalization and avoid a
    copy (if the input is already a numpy array).

clip : bool, default=False
    Set to True to clip transformed values of held-out data to
    provided `feature range`.

    .. versionadded:: 0.24

Attributes
----------
min_ : ndarray of shape (n_features,)
    Per feature adjustment for minimum. Equivalent to
    ``min - X.min(axis=0) * self.scale_``

scale_ : ndarray of shape (n_features,)
    Per feature relative scaling of the data. Equivalent to
    ``(max - min) / (X.max(axis=0) - X.min(axis=0))``

    .. versionadded:: 0.17
       *scale_* attribute.

data_min_ : ndarray of shape (n_features,)
    Per feature minimum seen in the data

    .. versionadded:: 0.17
       *data_min_*
```

```
data_max_ : ndarray of shape (n_features,)
    Per feature maximum seen in the data

    .. versionadded:: 0.17
       *data_max_*

data_range_ : ndarray of shape (n_features,)
    Per feature range ``(data_max_ - data_min_)`` seen in the data

    .. versionadded:: 0.17
       *data_range_*

n_features_in_ : int
    Number of features seen during :term:`fit`.

    .. versionadded:: 0.24

n_samples_seen_ : int
    The number of samples processed by the estimator.
    It will be reset on new calls to fit, but increments across
    ``partial_fit`` calls.

feature_names_in_ : ndarray of shape (`n_features_in_`,)
    Names of features seen during :term:`fit`. Defined only when `X`
    has feature names that are all strings.

    .. versionadded:: 1.0

See Also
--------
minmax_scale : Equivalent function without the estimator API.

Notes
-----
NaNs are treated as missing values: disregarded in fit, and maintained in
transform.

Examples
--------
>>> from sklearn.preprocessing import MinMaxScaler
>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
>>> scaler = MinMaxScaler()
>>> print(scaler.fit(data))
MinMaxScaler()
>>> print(scaler.data_max_)
[ 1. 18.]
>>> print(scaler.transform(data))
[[0.   0.  ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
>>> print(scaler.transform([[2, 2]]))
[[1.5 0. ]]
File:           c:\programdata\anaconda3\lib\site-packages\sklearn\preprocessing
\_data.py
Type:           type
Subclasses:
```

In [101…

```
scaler = MinMaxScaler()
```

In [102…
```
scaler.fit(raw_df[numeric_cols])
```

Out[102…
MinMaxScaler

▶ Parameters

In [103…
```
print('Minimum:')
list(scaler.data_min_)
```

```
Minimum:
```

Out[103…
```
[np.float64(-8.5),
 np.float64(-4.8),
 np.float64(0.0),
 np.float64(0.0),
 np.float64(0.0),
 np.float64(6.0),
 np.float64(0.0),
 np.float64(0.0),
 np.float64(0.0),
 np.float64(0.0),
 np.float64(980.5),
 np.float64(977.1),
 np.float64(0.0),
 np.float64(0.0),
 np.float64(-7.2),
 np.float64(-5.4)]
```

In [104…
```
print('Maximum:')
list(scaler.data_max_)
```

```
Maximum:
```

Out[104…
```
[np.float64(33.9),
 np.float64(48.1),
 np.float64(371.0),
 np.float64(145.0),
 np.float64(14.5),
 np.float64(135.0),
 np.float64(130.0),
 np.float64(87.0),
 np.float64(100.0),
 np.float64(100.0),
 np.float64(1041.0),
 np.float64(1039.6),
 np.float64(9.0),
 np.float64(9.0),
 np.float64(40.2),
 np.float64(46.7)]
```

We can now separately scale the training, validation and test sets using the `transform` method of `scaler`.

In [105…
```
train_inputs[numeric_cols]= scaler.transform(train_inputs[numeric_cols])
val_inputs[numeric_cols]= scaler.transform(val_inputs[numeric_cols])
test_inputs[numeric_cols]= scaler.transform(test_inputs[numeric_cols])
```

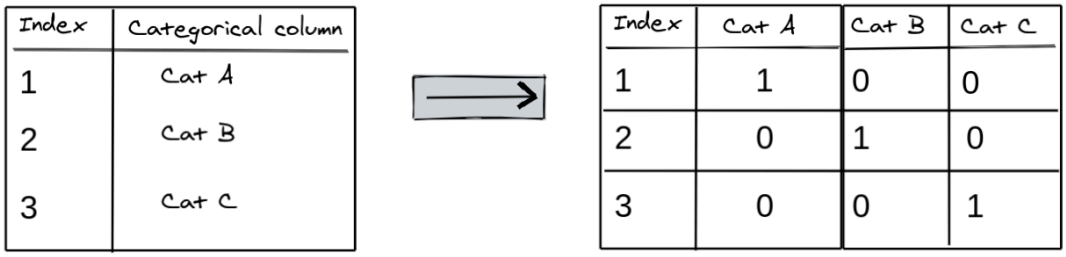
In [106…
```
train_inputs[numeric_cols].describe()
```

Out[106…

	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGust
count	97988.000000	97988.000000	97988.000000	97988.000000	97988.000000	97988.
mean	0.483689	0.525947	0.006396	0.036949	0.525366	0.
std	0.149458	0.131904	0.022962	0.021628	0.200931	0.
min	0.000000	0.013233	0.000000	0.000000	0.000000	0.
25%	0.377358	0.429112	0.000000	0.026207	0.517241	0.
50%	0.478774	0.514178	0.000000	0.037741	0.526244	0.
75%	0.591981	0.618147	0.002156	0.038621	0.634483	0.
max	1.000000	1.000000	1.000000	0.568276	0.986207	1.

Encoding Categorical Data

Since machine learning models can only be trained with numeric data, we need to convert categorical data to numbers. A common technique is to use one-hot encoding for categorical columns.



One hot encoding involves adding a new binary (0/1) column for each unique category of a categorical column.

In [107…
```
raw_df[categorical_cols].nunique()
```

Out[107…
```
Location       49
WindGustDir    16
WindDir9am     16
WindDir3pm     16
RainToday       2
dtype: int64
```

In [108…
```
from sklearn.preprocessing import OneHotEncoder
```

In [109…
```
?OneHotEncoder
```

```
Init signature:
OneHotEncoder(
    *,
    categories='auto',
    drop=None,
    sparse_output=True,
    dtype=<class 'numpy.float64'>,
    handle_unknown='error',
    min_frequency=None,
    max_categories=None,
    feature_name_combiner='concat',
)
Docstring:
Encode categorical features as a one-hot numeric array.

The input to this transformer should be an array-like of integers or
strings, denoting the values taken on by categorical (discrete) features.
The features are encoded using a one-hot (aka 'one-of-K' or 'dummy')
encoding scheme. This creates a binary column for each category and
returns a sparse matrix or dense array (depending on the ``sparse_output``
parameter).

By default, the encoder derives the categories based on the unique values
in each feature. Alternatively, you can also specify the `categories`
manually.

This encoding is needed for feeding categorical data to many scikit-learn
estimators, notably linear models and SVMs with the standard kernels.

Note: a one-hot encoding of y labels should use a LabelBinarizer
instead.

Read more in the :ref:`User Guide <preprocessing_categorical_features>`.
For a comparison of different encoders, refer to:
:ref:`sphx_glr_auto_examples_preprocessing_plot_target_encoder.py`.

Parameters
----------
categories : 'auto' or a list of array-like, default='auto'
    Categories (unique values) per feature:

    - 'auto' : Determine categories automatically from the training data.
    - list : ``categories[i]`` holds the categories expected in the ith
      column. The passed categories should not mix strings and numeric
      values within a single feature, and should be sorted in case of
      numeric values.

    The used categories can be found in the ``categories_`` attribute.

    .. versionadded:: 0.20

drop : {'first', 'if_binary'} or an array-like of shape (n_features,),
default=None
    Specifies a methodology to use to drop one of the categories per
    feature. This is useful in situations where perfectly collinear
    features cause problems, such as when feeding the resulting data
    into an unregularized linear regression model.

    However, dropping one category breaks the symmetry of the original
    representation and can therefore induce a bias in downstream models,
```

```
        for instance for penalized linear classification or regression models.

        - None : retain all features (the default).
        - 'first' : drop the first category in each feature. If only one
          category is present, the feature will be dropped entirely.
        - 'if_binary' : drop the first category in each feature with two
          categories. Features with 1 or more than 2 categories are
          left intact.
        - array : ``drop[i]`` is the category in feature ``X[:, i]`` that
          should be dropped.

        When `max_categories` or `min_frequency` is configured to group
        infrequent categories, the dropping behavior is handled after the
        grouping.

        .. versionadded:: 0.21
           The parameter `drop` was added in 0.21.

        .. versionchanged:: 0.23
           The option `drop='if_binary'` was added in 0.23.

        .. versionchanged:: 1.1
            Support for dropping infrequent categories.

    sparse_output : bool, default=True
        When ``True``, it returns a :class:`scipy.sparse.csr_matrix`,
        i.e. a sparse matrix in "Compressed Sparse Row" (CSR) format.

        .. versionadded:: 1.2
           `sparse` was renamed to `sparse_output`

    dtype : number type, default=np.float64
        Desired dtype of output.

    handle_unknown : {'error', 'ignore', 'infrequent_if_exist', 'warn'},
    default='error'
        Specifies the way unknown categories are handled during :meth:`transform`.

        - 'error' : Raise an error if an unknown category is present during transfor
    m.
        - 'ignore' : When an unknown category is encountered during
          transform, the resulting one-hot encoded columns for this feature
          will be all zeros. In the inverse transform, an unknown category
          will be denoted as None.
        - 'infrequent_if_exist' : When an unknown category is encountered
          during transform, the resulting one-hot encoded columns for this
          feature will map to the infrequent category if it exists. The
          infrequent category will be mapped to the last position in the
          encoding. During inverse transform, an unknown category will be
          mapped to the category denoted `'infrequent'` if it exists. If the
          `'infrequent'` category does not exist, then :meth:`transform` and
          :meth:`inverse_transform` will handle an unknown category as with
          `handle_unknown='ignore'`. Infrequent categories exist based on
          `min_frequency` and `max_categories`. Read more in the
          :ref:`User Guide <encoder_infrequent_categories>`.
        - 'warn' : When an unknown category is encountered during transform
          a warning is issued, and the encoding then proceeds as described for
          `handle_unknown="infrequent_if_exist"`.

        .. versionchanged:: 1.1
```

```
            ``'infrequent_if_exist'`` was added to automatically handle unknown
            categories and infrequent categories.

        .. versionadded:: 1.6
           The option `"warn"` was added in 1.6.

    min_frequency : int or float, default=None
        Specifies the minimum frequency below which a category will be
        considered infrequent.

        - If `int`, categories with a smaller cardinality will be considered
          infrequent.

        - If `float`, categories with a smaller cardinality than
          `min_frequency * n_samples`  will be considered infrequent.

        .. versionadded:: 1.1
            Read more in the :ref:`User Guide <encoder_infrequent_categories>`.

    max_categories : int, default=None
        Specifies an upper limit to the number of output features for each input
        feature when considering infrequent categories. If there are infrequent
        categories, `max_categories` includes the category representing the
        infrequent categories along with the frequent categories. If `None`,
        there is no limit to the number of output features.

        .. versionadded:: 1.1
            Read more in the :ref:`User Guide <encoder_infrequent_categories>`.

    feature_name_combiner : "concat" or callable, default="concat"
        Callable with signature `def callable(input_feature, category)` that returns
    a
        string. This is used to create feature names to be returned by
        :meth:`get_feature_names_out`.

        `"concat"` concatenates encoded feature name and category with
        `feature + "_" + str(category)`.E.g. feature X with values 1, 6, 7 create
        feature names `X_1, X_6, X_7`.

        .. versionadded:: 1.3

    Attributes
    ----------
    categories_ : list of arrays
        The categories of each feature determined during fitting
        (in order of the features in X and corresponding with the output
        of ``transform``). This includes the category specified in ``drop``
        (if any).

    drop_idx_ : array of shape (n_features,)
        - ``drop_idx_[i]`` is the index in ``categories_[i]`` of the category
          to be dropped for each feature.
        - ``drop_idx_[i] = None`` if no category is to be dropped from the
          feature with index ``i``, e.g. when `drop='if_binary'` and the
          feature isn't binary.
        - ``drop_idx_ = None`` if all the transformed features will be
          retained.

        If infrequent categories are enabled by setting `min_frequency` or
        `max_categories` to a non-default value and `drop_idx[i]` corresponds
```

```
    to a infrequent category, then the entire infrequent category is
    dropped.

    .. versionchanged:: 0.23
       Added the possibility to contain `None` values.

infrequent_categories_ : list of ndarray
    Defined only if infrequent categories are enabled by setting
    `min_frequency` or `max_categories` to a non-default value.
    `infrequent_categories_[i]` are the infrequent categories for feature
    `i`. If the feature `i` has no infrequent categories
    `infrequent_categories_[i]` is None.

    .. versionadded:: 1.1

n_features_in_ : int
    Number of features seen during :term:`fit`.

    .. versionadded:: 1.0

feature_names_in_ : ndarray of shape (`n_features_in_`,)
    Names of features seen during :term:`fit`. Defined only when `X`
    has feature names that are all strings.

    .. versionadded:: 1.0

feature_name_combiner : callable or None
    Callable with signature `def callable(input_feature, category)` that returns
a
    string. This is used to create feature names to be returned by
    :meth:`get_feature_names_out`.

    .. versionadded:: 1.3

See Also
--------
OrdinalEncoder : Performs an ordinal (integer)
  encoding of the categorical features.
TargetEncoder : Encodes categorical features using the target.
sklearn.feature_extraction.DictVectorizer : Performs a one-hot encoding of
  dictionary items (also handles string-valued features).
sklearn.feature_extraction.FeatureHasher : Performs an approximate one-hot
  encoding of dictionary items or strings.
LabelBinarizer : Binarizes labels in a one-vs-all
  fashion.
MultiLabelBinarizer : Transforms between iterable of
  iterables and a multilabel format, e.g. a (samples x classes) binary
  matrix indicating the presence of a class label.

Examples
--------
Given a dataset with two features, we let the encoder find the unique
values per feature and transform the data to a binary one-hot encoding.

>>> from sklearn.preprocessing import OneHotEncoder

One can discard categories not seen during `fit`:

>>> enc = OneHotEncoder(handle_unknown='ignore')
>>> X = [['Male', 1], ['Female', 3], ['Female', 2]]
```

```
>>> enc.fit(X)
OneHotEncoder(handle_unknown='ignore')
>>> enc.categories_
[array(['Female', 'Male'], dtype=object), array([1, 2, 3], dtype=object)]
>>> enc.transform([['Female', 1], ['Male', 4]]).toarray()
array([[1., 0., 1., 0., 0.],
       [0., 1., 0., 0., 0.]])
>>> enc.inverse_transform([[0, 1, 1, 0, 0], [0, 0, 0, 1, 0]])
array([['Male', 1],
       [None, 2]], dtype=object)
>>> enc.get_feature_names_out(['gender', 'group'])
array(['gender_Female', 'gender_Male', 'group_1', 'group_2', 'group_3'], ...)

One can always drop the first column for each feature:

>>> drop_enc = OneHotEncoder(drop='first').fit(X)
>>> drop_enc.categories_
[array(['Female', 'Male'], dtype=object), array([1, 2, 3], dtype=object)]
>>> drop_enc.transform([['Female', 1], ['Male', 2]]).toarray()
array([[0., 0., 0.],
       [1., 1., 0.]])

Or drop a column for feature only having 2 categories:

>>> drop_binary_enc = OneHotEncoder(drop='if_binary').fit(X)
>>> drop_binary_enc.transform([['Female', 1], ['Male', 2]]).toarray()
array([[0., 1., 0., 0.],
       [1., 0., 1., 0.]])

One can change the way feature names are created.

>>> def custom_combiner(feature, category):
...     return str(feature) + "_" + type(category).__name__ + "_" + str(category)
>>> custom_fnames_enc = OneHotEncoder(feature_name_combiner=custom_combiner).fit
(X)
>>> custom_fnames_enc.get_feature_names_out()
array(['x0_str_Female', 'x0_str_Male', 'x1_int_1', 'x1_int_2', 'x1_int_3'],
      dtype=object)

Infrequent categories are enabled by setting `max_categories` or `min_frequency`.

>>> import numpy as np
>>> X = np.array([["a"] * 5 + ["b"] * 20 + ["c"] * 10 + ["d"] * 3], dtype=objec
t).T
>>> ohe = OneHotEncoder(max_categories=3, sparse_output=False).fit(X)
>>> ohe.infrequent_categories_
[array(['a', 'd'], dtype=object)]
>>> ohe.transform([["a"], ["b"]])
array([[0., 0., 1.],
       [1., 0., 0.]])
File:           c:\programdata\anaconda3\lib\site-packages\sklearn\preprocessing
\_encoders.py
Type:           type
Subclasses:
```

In [111…
```
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
```

In [112…
```
encoder.fit(raw_df[categorical_cols])
```

Out[112…

OneHotEncoder

▶ Parameters

In [113…
```
encoder.categories_
```

Out[113…
```
[array(['Adelaide', 'Albany', 'Albury', 'AliceSprings', 'BadgerysCreek',
        'Ballarat', 'Bendigo', 'Brisbane', 'Cairns', 'Canberra', 'Cobar',
        'CoffsHarbour', 'Dartmoor', 'Darwin', 'GoldCoast', 'Hobart',
        'Katherine', 'Launceston', 'Melbourne', 'MelbourneAirport',
        'Mildura', 'Moree', 'MountGambier', 'MountGinini', 'Newcastle',
        'Nhil', 'NorahHead', 'NorfolkIsland', 'Nuriootpa', 'PearceRAAF',
        'Penrith', 'Perth', 'PerthAirport', 'Portland', 'Richmond', 'Sale',
        'SalmonGums', 'Sydney', 'SydneyAirport', 'Townsville',
        'Tuggeranong', 'Uluru', 'WaggaWagga', 'Walpole', 'Watsonia',
        'Williamtown', 'Witchcliffe', 'Wollongong', 'Woomera'],
       dtype=object),
 array(['E', 'ENE', 'ESE', 'N', 'NE', 'NNE', 'NNW', 'NW', 'S', 'SE', 'SSE',
        'SSW', 'SW', 'W', 'WNW', 'WSW', nan], dtype=object),
 array(['E', 'ENE', 'ESE', 'N', 'NE', 'NNE', 'NNW', 'NW', 'S', 'SE', 'SSE',
        'SSW', 'SW', 'W', 'WNW', 'WSW', nan], dtype=object),
 array(['E', 'ENE', 'ESE', 'N', 'NE', 'NNE', 'NNW', 'NW', 'S', 'SE', 'SSE',
        'SSW', 'SW', 'W', 'WNW', 'WSW', nan], dtype=object),
 array(['No', 'Yes'], dtype=object)]
```

In [115…
```
encoded_cols = list(encoder.get_feature_names_out(categorical_cols))
print(encoded_cols)
```

```
['Location_Adelaide', 'Location_Albany', 'Location_Albury', 'Location_AliceSpring
s', 'Location_BadgerysCreek', 'Location_Ballarat', 'Location_Bendigo', 'Location_
Brisbane', 'Location_Cairns', 'Location_Canberra', 'Location_Cobar', 'Location_Co
ffsHarbour', 'Location_Dartmoor', 'Location_Darwin', 'Location_GoldCoast', 'Locat
ion_Hobart', 'Location_Katherine', 'Location_Launceston', 'Location_Melbourne',
'Location_MelbourneAirport', 'Location_Mildura', 'Location_Moree', 'Location_Moun
tGambier', 'Location_MountGinini', 'Location_Newcastle', 'Location_Nhil', 'Locati
on_NorahHead', 'Location_NorfolkIsland', 'Location_Nuriootpa', 'Location_PearceRA
AF', 'Location_Penrith', 'Location_Perth', 'Location_PerthAirport', 'Location_Por
tland', 'Location_Richmond', 'Location_Sale', 'Location_SalmonGums', 'Location_Sy
dney', 'Location_SydneyAirport', 'Location_Townsville', 'Location_Tuggeranong',
'Location_Uluru', 'Location_WaggaWagga', 'Location_Walpole', 'Location_Watsonia',
'Location_Williamtown', 'Location_Witchcliffe', 'Location_Wollongong', 'Location_
Woomera', 'WindGustDir_E', 'WindGustDir_ENE', 'WindGustDir_ESE', 'WindGustDir_N',
'WindGustDir_NE', 'WindGustDir_NNE', 'WindGustDir_NNW', 'WindGustDir_NW', 'WindGu
stDir_S', 'WindGustDir_SE', 'WindGustDir_SSE', 'WindGustDir_SSW', 'WindGustDir_S
W', 'WindGustDir_W', 'WindGustDir_WNW', 'WindGustDir_WSW', 'WindGustDir_nan', 'Wi
ndDir9am_E', 'WindDir9am_ENE', 'WindDir9am_ESE', 'WindDir9am_N', 'WindDir9am_NE',
'WindDir9am_NNE', 'WindDir9am_NNW', 'WindDir9am_NW', 'WindDir9am_S', 'WindDir9am_
SE', 'WindDir9am_SSE', 'WindDir9am_SSW', 'WindDir9am_SW', 'WindDir9am_W', 'WindDi
r9am_WNW', 'WindDir9am_WSW', 'WindDir9am_nan', 'WindDir3pm_E', 'WindDir3pm_ENE',
'WindDir3pm_ESE', 'WindDir3pm_N', 'WindDir3pm_NE', 'WindDir3pm_NNE', 'WindDir3pm_
NNW', 'WindDir3pm_NW', 'WindDir3pm_S', 'WindDir3pm_SE', 'WindDir3pm_SSE', 'WindDi
r3pm_SSW', 'WindDir3pm_SW', 'WindDir3pm_W', 'WindDir3pm_WNW', 'WindDir3pm_WSW',
'WindDir3pm_nan', 'RainToday_No', 'RainToday_Yes']
```

In [116…
```
train_inputs[encoded_cols] = encoder.transform(train_inputs[categorical_cols])
val_inputs[encoded_cols] = encoder.transform(val_inputs[categorical_cols])
test_inputs[encoded_cols] = encoder.transform(test_inputs[categorical_cols])
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
```

```
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:1: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
```

```
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:2: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
```

```
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
```

```
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\1584174743.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

In [117…

```
pd.set_option('display.max_columns', None)
```

In [118…

```
test_inputs
```

Out[118...

	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir
2498	Albury	0.681604	0.801512	0.000000	0.037741	0.526244	ENE
2499	Albury	0.693396	0.725898	0.001078	0.037741	0.526244	SSE
2500	Albury	0.634434	0.527410	0.005930	0.037741	0.526244	ENE
2501	Albury	0.608491	0.538752	0.042049	0.037741	0.526244	SSE
2502	Albury	0.566038	0.523629	0.018329	0.037741	0.526244	ENE
...	...	...	...	...	...	...	...
145454	Uluru	0.283019	0.502836	0.000000	0.037741	0.526244	E
145455	Uluru	0.266509	0.533081	0.000000	0.037741	0.526244	E
145456	Uluru	0.285377	0.568998	0.000000	0.037741	0.526244	NNW
145457	Uluru	0.327830	0.599244	0.000000	0.037741	0.526244	N
145458	Uluru	0.384434	0.601134	0.000000	0.037741	0.526244	SE

25710 rows × 123 columns

In [119...

```
print('train_inputs:', train_inputs.shape)
print('train_targets:', train_targets.shape)
print('val_inputs:', val_inputs.shape)
print('val_targets:', val_targets.shape)
print('test_inputs:', test_inputs.shape)
print('test_targets:', test_targets.shape)
```

```
train_inputs: (97988, 123)
train_targets: (97988,)
val_inputs: (17089, 123)
val_targets: (17089,)
test_inputs: (25710, 123)
test_targets: (25710,)
```

In [120...

```
!pip install pyarrow --quiet
```

In [121...

```
train_inputs.to_parquet('train_inputs.parquet')
val_inputs.to_parquet('val_inputs.parquet')
test_inputs.to_parquet('test_inputs.parquet')
```

In [122...

```
%%time
pd.DataFrame(train_targets).to_parquet('train_targets.parquet')
pd.DataFrame(val_targets).to_parquet('val_targets.parquet')
pd.DataFrame(test_targets).to_parquet('test_targets.parquet')
```

```
CPU times: total: 62.5 ms
Wall time: 101 ms
```

In [123...

```
%%time

train_inputs = pd.read_parquet('train_inputs.parquet')
val_inputs = pd.read_parquet('val_inputs.parquet')
test_inputs = pd.read_parquet('test_inputs.parquet')
```

```
train_targets = pd.read_parquet('train_targets.parquet')[target_col]
val_targets = pd.read_parquet('val_targets.parquet')[target_col]
test_targets = pd.read_parquet('test_targets.parquet')[target_col]
```

```
CPU times: total: 1.19 s
Wall time: 898 ms
```

In [124…

```
print('train_inputs:', train_inputs.shape)
print('train_targets:', train_targets.shape)
print('val_inputs:', val_inputs.shape)
print('val_targets:', val_targets.shape)
print('test_inputs:', test_inputs.shape)
print('test_targets:', test_targets.shape)
```

```
train_inputs: (97988, 123)
train_targets: (97988,)
val_inputs: (17089, 123)
val_targets: (17089,)
test_inputs: (25710, 123)
test_targets: (25710,)
```

In [125…

```
val_inputs
```

Out[125…

	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir
2133	Albury	0.469340	0.724008	0.0	0.037741	0.526244	WSW
2134	Albury	0.566038	0.839319	0.0	0.037741	0.526244	NE
2135	Albury	0.603774	0.814745	0.0	0.037741	0.526244	NNE
2136	Albury	0.813679	0.716446	0.0	0.037741	0.526244	NNE
2137	Albury	0.648585	0.756144	0.0	0.037741	0.526244	E
…	…	…	…	…	…	…	…
144913	Uluru	0.683962	0.746692	0.0	0.037741	0.526244	E
144914	Uluru	0.625000	0.778828	0.0	0.037741	0.526244	ESE
144915	Uluru	0.613208	0.792060	0.0	0.037741	0.526244	E
144916	Uluru	0.672170	0.826087	0.0	0.037741	0.526244	E
144917	Uluru	0.655660	0.797732	0.0	0.037741	0.526244	SE

17089 rows × 123 columns

In [126…

```
val_targets
```

Out[126…

```
2133      No
2134      No
2135      No
2136      No
2137      No
          ..
144913    No
144914    No
144915    No
144916    No
144917    No
Name: RainTomorrow, Length: 17089, dtype: object
```

Training a Logistic Regression Model

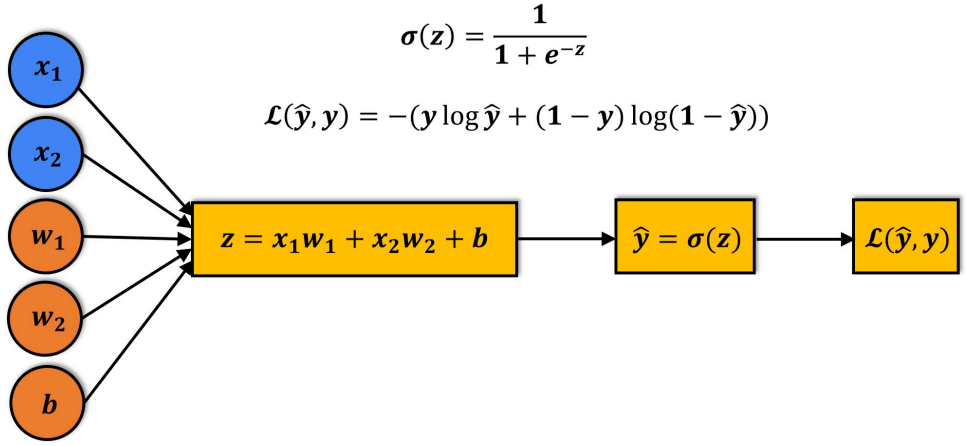
Logistic regression is a commonly used technique for solving binary classification problems. In a logistic regression model:

- we take linear combination (or weighted sum of the input features)
- we apply the sigmoid function to the result to obtain a number between 0 and 1
- this number represents the probability of the input being classified as "Yes"
- instead of RMSE, the cross entropy loss function is used to evaluate the results

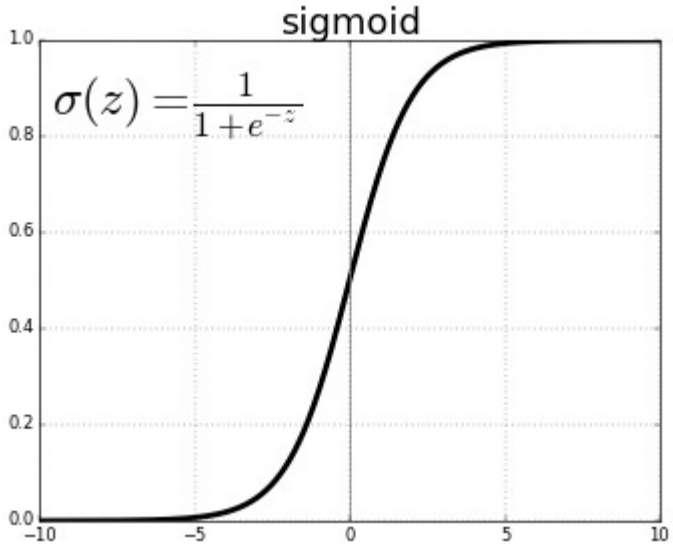
Here's a visual summary of how a logistic regression model is structured (source):

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1-y)\log(1-\hat{y}))$$



The sigmoid function applied to the linear combination of inputs has the following formula:



To train a logistic regression model, we can use the `LogisticRegression` class from Scikit-learn.

In [127…
```
from sklearn.linear_model import LogisticRegression
```

In [128…
```
?LogisticRegression
```

```
Init signature:
LogisticRegression(
    penalty='l2',
    *,
    dual=False,
    tol=0.0001,
    C=1.0,
    fit_intercept=True,
    intercept_scaling=1,
    class_weight=None,
    random_state=None,
    solver='lbfgs',
    max_iter=100,
    multi_class='deprecated',
    verbose=0,
    warm_start=False,
    n_jobs=None,
    l1_ratio=None,
)
Docstring:
Logistic Regression (aka logit, MaxEnt) classifier.

This class implements regularized logistic regression using the
'liblinear' library, 'newton-cg', 'sag', 'saga' and 'lbfgs' solvers. **Note
that regularization is applied by default**. It can handle both dense
and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit
floats for optimal performance; any other input format will be converted
(and copied).

The 'newton-cg', 'sag', and 'lbfgs' solvers support only L2 regularization
with primal formulation, or no regularization. The 'liblinear' solver
supports both L1 and L2 regularization, with a dual formulation only for
the L2 penalty. The Elastic-Net regularization is only supported by the
'saga' solver.

For :term:`multiclass` problems, all solvers but 'liblinear' optimize the
(penalized) multinomial loss. 'liblinear' only handle binary classification but c
an
be extended to handle multiclass by using
:class:`~sklearn.multiclass.OneVsRestClassifier`.

Read more in the :ref:`User Guide <logistic_regression>`.

Parameters
----------
penalty : {'l1', 'l2', 'elasticnet', None}, default='l2'
    Specify the norm of the penalty:

    - `None`: no penalty is added;
    - `'l2'`: add a L2 penalty term and it is the default choice;
    - `'l1'`: add a L1 penalty term;
    - `'elasticnet'`: both L1 and L2 penalty terms are added.

    .. warning::
       Some penalties may not work with some solvers. See the parameter
       `solver` below, to know the compatibility between the penalty and
       solver.

    .. versionadded:: 0.19
       l1 penalty with SAGA solver (allowing 'multinomial' + L1)
```

```
    dual : bool, default=False
        Dual (constrained) or primal (regularized, see also
        :ref:`this equation <regularized-logistic-loss>`) formulation. Dual formulati
    on
        is only implemented for l2 penalty with liblinear solver. Prefer dual=False w
    hen
        n_samples > n_features.

    tol : float, default=1e-4
        Tolerance for stopping criteria.

    C : float, default=1.0
        Inverse of regularization strength; must be a positive float.
        Like in support vector machines, smaller values specify stronger
        regularization.

    fit_intercept : bool, default=True
        Specifies if a constant (a.k.a. bias or intercept) should be
        added to the decision function.

    intercept_scaling : float, default=1
        Useful only when the solver `liblinear` is used
        and `self.fit_intercept` is set to `True`. In this case, `x` becomes
        `[x, self.intercept_scaling]`,
        i.e. a "synthetic" feature with constant value equal to
        `intercept_scaling` is appended to the instance vector.
        The intercept becomes
        ``intercept_scaling * synthetic_feature_weight``.

        .. note::
            The synthetic feature weight is subject to L1 or L2
            regularization as all other features.
            To lessen the effect of regularization on synthetic feature weight
            (and therefore on the intercept) `intercept_scaling` has to be increased.

    class_weight : dict or 'balanced', default=None
        Weights associated with classes in the form ``{class_label: weight}``.
        If not given, all classes are supposed to have weight one.

        The "balanced" mode uses the values of y to automatically adjust
        weights inversely proportional to class frequencies in the input data
        as ``n_samples / (n_classes * np.bincount(y))``.

        Note that these weights will be multiplied with sample_weight (passed
        through the fit method) if sample_weight is specified.

        .. versionadded:: 0.17
           *class_weight='balanced'*

    random_state : int, RandomState instance, default=None
        Used when ``solver`` == 'sag', 'saga' or 'liblinear' to shuffle the
        data. See :term:`Glossary <random_state>` for details.

    solver : {'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'},
    default='lbfgs'

        Algorithm to use in the optimization problem. Default is 'lbfgs'.
        To choose a solver, you might want to consider the following aspects:
```

```
    - For small datasets, 'liblinear' is a good choice, whereas 'sag'
      and 'saga' are faster for large ones;
    - For :term:`multiclass` problems, all solvers except 'liblinear' minimize th
e
      full multinomial loss;
    - 'liblinear' can only handle binary classification by default. To apply a
      one-versus-rest scheme for the multiclass setting one can wrap it with the
      :class:`~sklearn.multiclass.OneVsRestClassifier`.
    - 'newton-cholesky' is a good choice for
      `n_samples` >> `n_features * n_classes`, especially with one-hot encoded
      categorical features with rare categories. Be aware that the memory usage
      of this solver has a quadratic dependency on `n_features * n_classes`
      because it explicitly computes the full Hessian matrix.

    .. warning::
       The choice of the algorithm depends on the penalty chosen and on
       (multinomial) multiclass support:

       ================ ============================== ======================
       solver           penalty                        multinomial multiclass
       ================ ============================== ======================
       'lbfgs'          'l2', None                     yes
       'liblinear'      'l1', 'l2'                     no
       'newton-cg'      'l2', None                     yes
       'newton-cholesky' 'l2', None                    yes
       'sag'            'l2', None                     yes
       'saga'           'elasticnet', 'l1', 'l2', None yes
       ================ ============================== ======================

    .. note::
       'sag' and 'saga' fast convergence is only guaranteed on features
       with approximately the same scale. You can preprocess the data with
       a scaler from :mod:`sklearn.preprocessing`.

    .. seealso::
       Refer to the :ref:`User Guide <Logistic_regression>` for more
       information regarding :class:`LogisticRegression` and more specifically th
e
       :ref:`Table <logistic_regression_solvers>`
       summarizing solver/penalty supports.

    .. versionadded:: 0.17
       Stochastic Average Gradient (SAG) descent solver. Multinomial support in
       version 0.18.
    .. versionadded:: 0.19
       SAGA solver.
    .. versionchanged:: 0.22
       The default solver changed from 'liblinear' to 'lbfgs' in 0.22.
    .. versionadded:: 1.2
       newton-cholesky solver. Multinomial support in version 1.6.

max_iter : int, default=100
    Maximum number of iterations taken for the solvers to converge.

multi_class : {'auto', 'ovr', 'multinomial'}, default='auto'
    If the option chosen is 'ovr', then a binary problem is fit for each
    label. For 'multinomial' the loss minimised is the multinomial loss fit
    across the entire probability distribution, *even when the data is
    binary*. 'multinomial' is unavailable when solver='liblinear'.
    'auto' selects 'ovr' if the data is binary, or if solver='liblinear',
```

```
    and otherwise selects 'multinomial'.

    .. versionadded:: 0.18
       Stochastic Average Gradient descent solver for 'multinomial' case.
    .. versionchanged:: 0.22
        Default changed from 'ovr' to 'auto' in 0.22.
    .. deprecated:: 1.5
       ``multi_class`` was deprecated in version 1.5 and will be removed in 1.8.
       From then on, the recommended 'multinomial' will always be used for
       `n_classes >= 3`.
       Solvers that do not support 'multinomial' will raise an error.
       Use `sklearn.multiclass.OneVsRestClassifier(LogisticRegression())` if you
       still want to use OvR.

verbose : int, default=0
    For the liblinear and lbfgs solvers set verbose to any positive
    number for verbosity.

warm_start : bool, default=False
    When set to True, reuse the solution of the previous call to fit as
    initialization, otherwise, just erase the previous solution.
    Useless for liblinear solver. See :term:`the Glossary <warm_start>`.

    .. versionadded:: 0.17
       *warm_start* to support *lbfgs*, *newton-cg*, *sag*, *saga* solvers.

n_jobs : int, default=None
    Number of CPU cores used when parallelizing over classes if
    multi_class='ovr'". This parameter is ignored when the ``solver`` is
    set to 'liblinear' regardless of whether 'multi_class' is specified or
    not. ``None`` means 1 unless in a :obj:`joblib.parallel_backend`
    context. ``-1`` means using all processors.
    See :term:`Glossary <n_jobs>` for more details.

l1_ratio : float, default=None
    The Elastic-Net mixing parameter, with ``0 <= l1_ratio <= 1``. Only
    used if ``penalty='elasticnet'``. Setting ``l1_ratio=0`` is equivalent
    to using ``penalty='l2'``, while setting ``l1_ratio=1`` is equivalent
    to using ``penalty='l1'``. For ``0 < l1_ratio <1``, the penalty is a
    combination of L1 and L2.

Attributes
----------

classes_ : ndarray of shape (n_classes, )
    A list of class labels known to the classifier.

coef_ : ndarray of shape (1, n_features) or (n_classes, n_features)
    Coefficient of the features in the decision function.

    `coef_` is of shape (1, n_features) when the given problem is binary.
    In particular, when `multi_class='multinomial'`, `coef_` corresponds
    to outcome 1 (True) and `-coef_` corresponds to outcome 0 (False).

intercept_ : ndarray of shape (1,) or (n_classes,)
    Intercept (a.k.a. bias) added to the decision function.

    If `fit_intercept` is set to False, the intercept is set to zero.
    `intercept_` is of shape (1,) when the given problem is binary.
    In particular, when `multi_class='multinomial'`, `intercept_`
```

```
    corresponds to outcome 1 (True) and `-intercept_` corresponds to
    outcome 0 (False).

n_features_in_ : int
    Number of features seen during :term:`fit`.

    .. versionadded:: 0.24

feature_names_in_ : ndarray of shape (`n_features_in_`,)
    Names of features seen during :term:`fit`. Defined only when `X`
    has feature names that are all strings.

    .. versionadded:: 1.0

n_iter_ : ndarray of shape (n_classes,) or (1, )
    Actual number of iterations for all classes. If binary or multinomial,
    it returns only 1 element. For liblinear solver, only the maximum
    number of iteration across all classes is given.

    .. versionchanged:: 0.20

        In SciPy <= 1.0.0 the number of lbfgs iterations may exceed
        ``max_iter``. ``n_iter_`` will now report at most ``max_iter``.

See Also
--------
SGDClassifier : Incrementally trained logistic regression (when given
    the parameter ``loss="log_loss"``).
LogisticRegressionCV : Logistic regression with built-in cross validation.

Notes
-----
The underlying C implementation uses a random number generator to
select features when fitting the model. It is thus not uncommon,
to have slightly different results for the same input data. If
that happens, try with a smaller tol parameter.

Predict output may not match that of standalone liblinear in certain
cases. See :ref:`differences from liblinear <liblinear_differences>`
in the narrative documentation.

References
----------

L-BFGS-B -- Software for Large-scale Bound-constrained Optimization
    Ciyou Zhu, Richard Byrd, Jorge Nocedal and Jose Luis Morales.
    http://users.iems.northwestern.edu/~nocedal/lbfgsb.html

LIBLINEAR -- A Library for Large Linear Classification
    https://www.csie.ntu.edu.tw/~cjlin/liblinear/

SAG -- Mark Schmidt, Nicolas Le Roux, and Francis Bach
    Minimizing Finite Sums with the Stochastic Average Gradient
    https://hal.inria.fr/hal-00860051/document

SAGA -- Defazio, A., Bach F. & Lacoste-Julien S. (2014).
        :arxiv:`"SAGA: A Fast Incremental Gradient Method With Support
        for Non-Strongly Convex Composite Objectives" <1407.0202>`

Hsiang-Fu Yu, Fang-Lan Huang, Chih-Jen Lin (2011). Dual coordinate descent
```

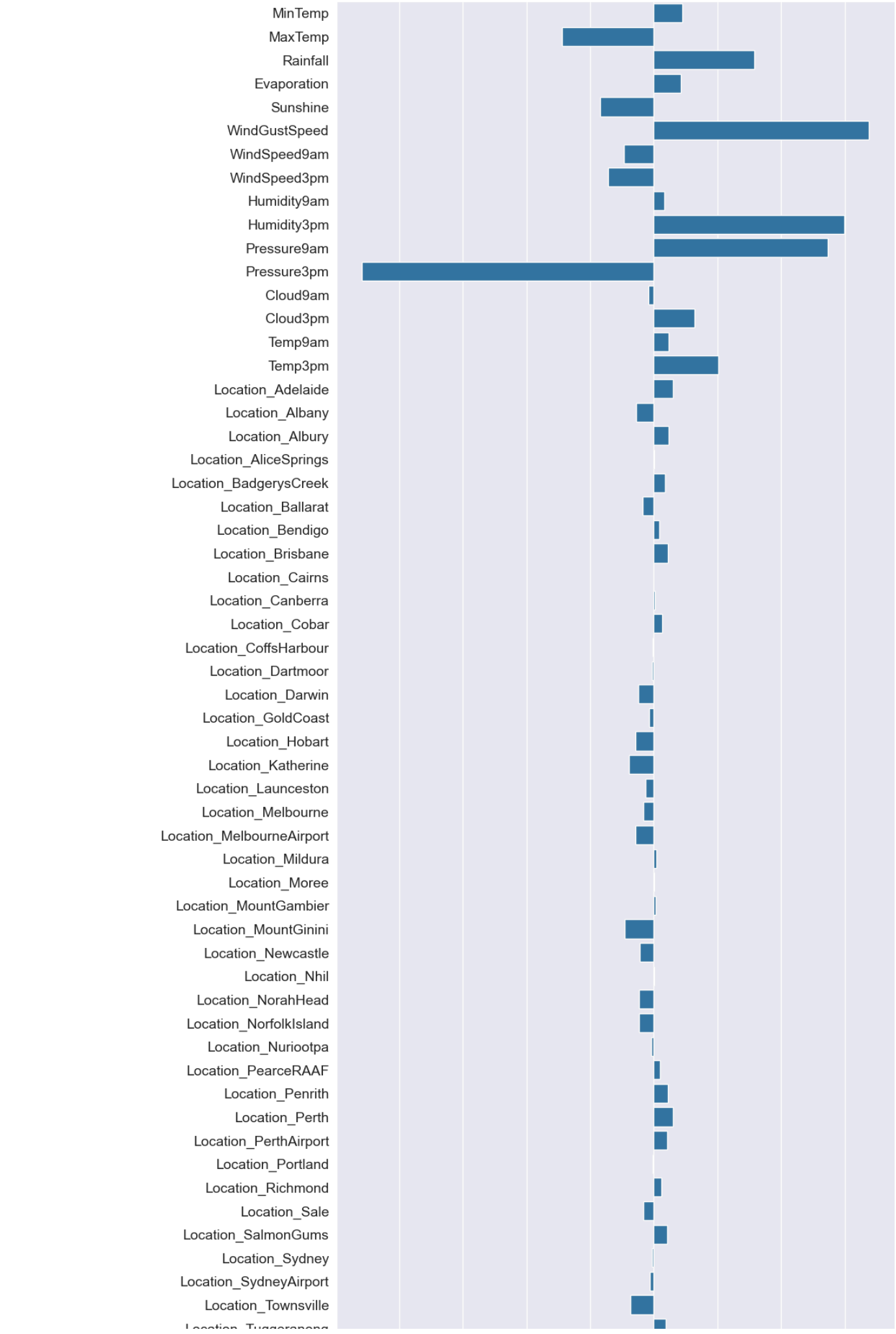
```
    methods for logistic regression and maximum entropy models.
    Machine Learning 85(1-2):41-75.
    https://www.csie.ntu.edu.tw/~cjlin/papers/maxent_dual.pdf

Examples
--------
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.82e-01, 1.82e-02, 1.44e-08],
       [9.72e-01, 2.82e-02, 3.02e-08]])
>>> clf.score(X, y)
0.97

For a comparison of the LogisticRegression with other classifiers see:
:ref:`sphx_glr_auto_examples_classification_plot_classification_probability.py`.
File:           c:\programdata\anaconda3\lib\site-packages\sklearn\linear_model\_
logistic.py
Type:           type
Subclasses:     LogisticRegressionCV
```

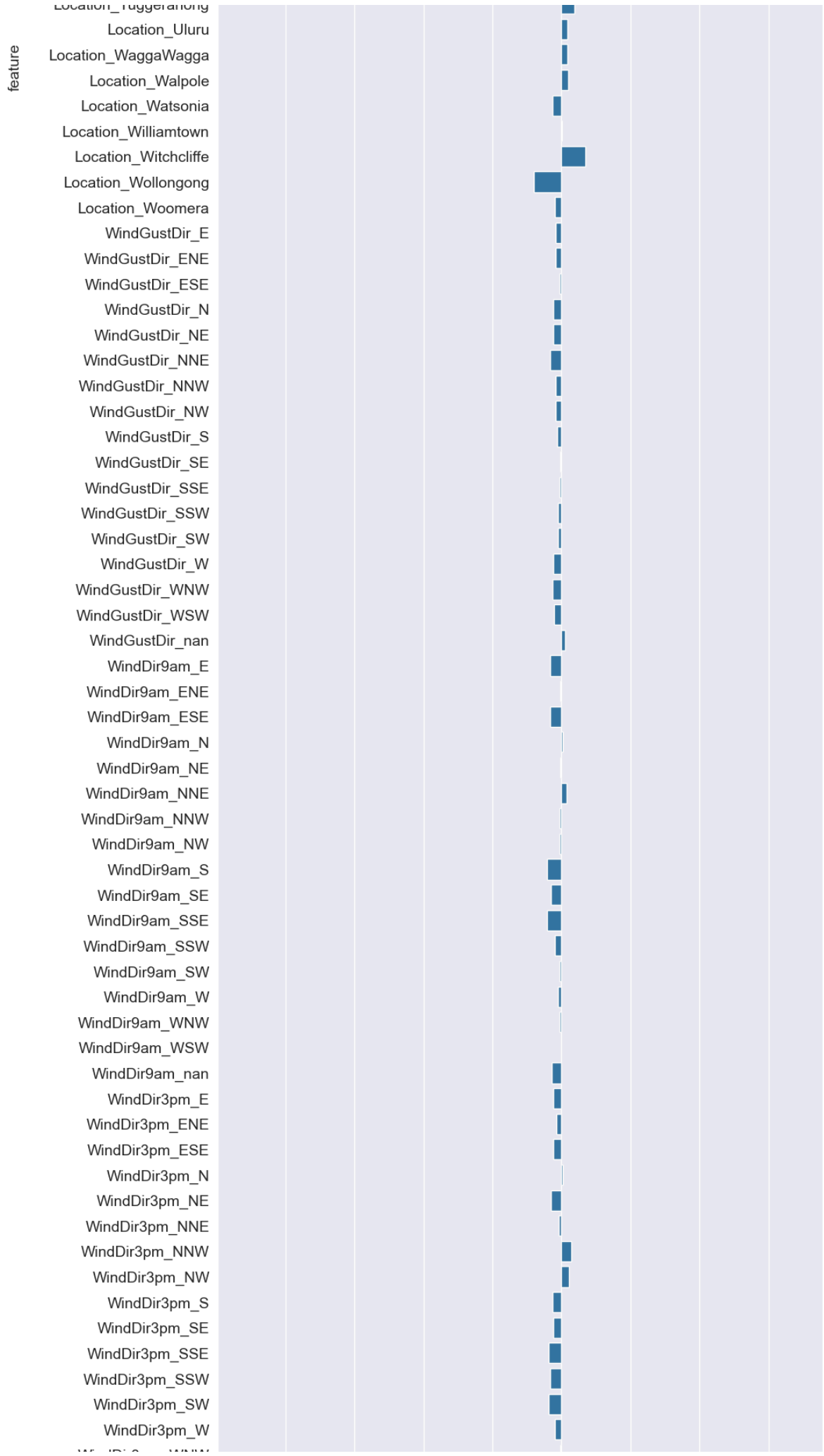
In [137…
```
weight_df = pd.DataFrame({'feature':(numeric_cols + encoded_cols),'weight':model
```

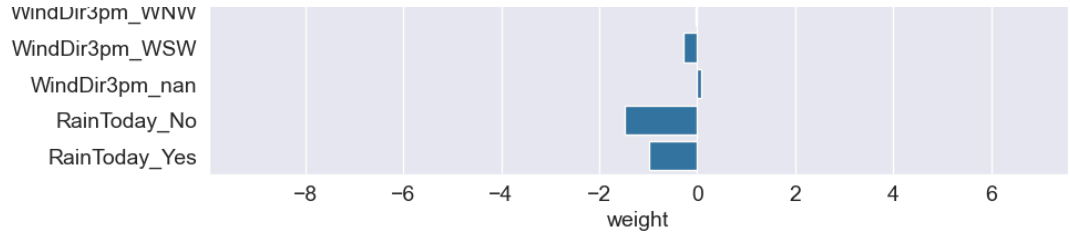
In [141…
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [143…
```
plt.figure(figsize=(10, 50))
sns.barplot(data= weight_df,x='weight',y='feature')
```

Out[143…
```
<Axes: xlabel='weight', ylabel='feature'>
```


MinTemp
MaxTemp
Rainfall
Evaporation
Sunshine
WindGustSpeed
WindSpeed9am
WindSpeed3pm
Humidity9am
Humidity3pm
Pressure9am
Pressure3pm
Cloud9am
Cloud3pm
Temp9am
Temp3pm
Location_Adelaide
Location_Albany
Location_Albury
Location_AliceSprings
Location_BadgerysCreek
Location_Ballarat
Location_Bendigo
Location_Brisbane
Location_Cairns
Location_Canberra
Location_Cobar
Location_CoffsHarbour
Location_Dartmoor
Location_Darwin
Location_GoldCoast
Location_Hobart
Location_Katherine
Location_Launceston
Location_Melbourne
Location_MelbourneAirport
Location_Mildura
Location_Moree
Location_MountGambier
Location_MountGinini
Location_Newcastle
Location_Nhil
Location_NorahHead
Location_NorfolkIsland
Location_Nuriootpa
Location_PearceRAAF
Location_Penrith
Location_Perth
Location_PerthAirport
Location_Portland
Location_Richmond
Location_Sale
Location_SalmonGums
Location_Sydney
Location_SydneyAirport
Location_Townsville


feature
Location_Uluru
Location_WaggaWagga
Location_Walpole
Location_Watsonia
Location_Williamtown
Location_Witchcliffe
Location_Wollongong
Location_Woomera
WindGustDir_E
WindGustDir_ENE
WindGustDir_ESE
WindGustDir_N
WindGustDir_NE
WindGustDir_NNE
WindGustDir_NNW
WindGustDir_NW
WindGustDir_S
WindGustDir_SE
WindGustDir_SSE
WindGustDir_SSW
WindGustDir_SW
WindGustDir_W
WindGustDir_WNW
WindGustDir_WSW
WindGustDir_nan
WindDir9am_E
WindDir9am_ENE
WindDir9am_ESE
WindDir9am_N
WindDir9am_NE
WindDir9am_NNE
WindDir9am_NNW
WindDir9am_NW
WindDir9am_S
WindDir9am_SE
WindDir9am_SSE
WindDir9am_SSW
WindDir9am_SW
WindDir9am_W
WindDir9am_WNW
WindDir9am_WSW
WindDir9am_nan
WindDir3pm_E
WindDir3pm_ENE
WindDir3pm_ESE
WindDir3pm_N
WindDir3pm_NE
WindDir3pm_NNE
WindDir3pm_NNW
WindDir3pm_NW
WindDir3pm_S
WindDir3pm_SE
WindDir3pm_SSE
WindDir3pm_SSW
WindDir3pm_SW
WindDir3pm_W



In [129…
```
model = LogisticRegression(solver='liblinear')
```

In [130…
```
model.fit(train_inputs[numeric_cols + encoded_cols], train_targets)
```

Out[130…

▾ LogisticRegression

▶ Parameters

In [131…
```
print(model.coef_.tolist())
```

```
[[0.8986309524307585, -2.8799139603656476, 3.1627778792925136, 0.8542480325512375, -1.6713937552831495, 6.764402793259662, -0.9423225980610669, -1.428429341415226, 0.3228916345620227, 5.995314595845687, 5.46385798681765, -9.176805495017025, -0.16229605212992387, 1.2876601324108778, 0.47471552862801175, 2.0214286626567652, 0.601649894260212, -0.5524828302537507, 0.478142525141566, 0.0076701621765451955, 0.3468143810158331, -0.35227684051730773, 0.17971046554356387, 0.44048611891207949 6, -0.013981965366035779, 0.028943930135756368, 0.25814751427032123, -0.021205184295877064, -0.04279543372469812, -0.48314187955522264, -0.13756306191765827, -0.5760590123158442, -0.7875244847653909, -0.2554044687914849, -0.32888336386240463, -0.5690038402631256, 0.08183005785914067, 0.013382331499554506, 0.064127651106477, -0.9020544131910037, -0.44433028149664067, 0.008516524825669197, -0.4606124060449102, -0.46551787105596615, -0.0694985463047282, 0.19115891731761225, 0.4504755738364723, 0.6081211328747446, 0.4273140143056061, -0.028331114588813006, 0.2515462690728239, -0.3216053754036128, 0.42495610270699585, -0.059037289201670576, -0.11319964939006641, -0.728376704769763, 0.3664523204907733, 0.18359063890471336, 0.18397520858567054, 0.1866035170437098, -0.24926976323695757, 0.017948527756354016, 0.7034022078724789, -0.8000206172370024, -0.19234376509203757, -0.16179772106284168, -0.1589856246170296, -0.06310025737578466, -0.22355076319571363, -0.2323920515637488, -0.32115335381344245, -0.16556977881162122, -0.15884877230023461, -0.1109763837129809, -0.03252775564551439, -0.050627842901789157, -0.09419866719646963, -0.09081634215743242, -0.22315610384273749, -0.2585278791742812, -0.20081549362194173, 0.09749068607475808, -0.31877393396352055, -0.02935665639365822, -0.3286133150231324, 0.029865119285056665, -0.021068357129804854, 0.14405278299130433, -0.06176891429627259, -0.056973933693723604, -0.4085341814329305, -0.308123062829233, -0.40543593127839783, -0.19482797568172502, -0.06029128280580611, -0.086920761240314, -0.0571113352318563, -0.01613042203007941, -0.2695419441647847, -0.23985345234523577246, -0.15250072707399268, -0.23483449046146043, 0.038196930872560174, -0.31141270858051723, -0.08024552240056207, 0.284544247003666, 0.22185082268005635, -0.26544832088457293, -0.24117418904119328, -0.36688057179854594, -0.3168627691697572, -0.370021328871069, -0.18037063315712984, -0.03349918929332493, -0.2759753696917143, 0.07493316730871326, -1.4735166756087341, -0.9760374293361063]]
```

In [132…
```
print(model.intercept_)
```

```
[-2.4495541]
```

In [133…
```
X_train = train_inputs[numeric_cols + encoded_cols]
X_val = val_inputs[numeric_cols + encoded_cols]
X_test = test_inputs[numeric_cols + encoded_cols]
```

In [134…
```
train_preds = model.predict(X_train)
```

In [136…
```
train_preds
```

Out[136…
```
array(['No', 'No', 'No', ..., 'No', 'No', 'No'],
      shape=(97988,), dtype=object)
```

Making Predictions and Evaluating the Model

We can now use the trained model to make predictions on the training, test

In [144…
```
X_train = train_inputs[numeric_cols + encoded_cols]
X_val = val_inputs[numeric_cols + encoded_cols]
X_test = test_inputs[numeric_cols + encoded_cols]
```

In [145…
```
train_preds = model.predict(X_train)
```

In [146…
```
train_preds
```

Out[146…
```
array(['No', 'No', 'No', ..., 'No', 'No', 'No'],
      shape=(97988,), dtype=object)
```

In [147…
```
train_targets
```

Out[147…
```
0         No
1         No
2         No
3         No
4         No
          ..
144548    No
144549    No
144550    No
144551    No
144552    No
Name: RainTomorrow, Length: 97988, dtype: object
```

In [148…
```
train_probs = model.predict_proba(X_train)
```

In [149…
```
train_probs
```

Out[149…
```
array([[0.94401103, 0.05598897],
       [0.9407411 , 0.0592589 ],
       [0.96093594, 0.03906406],
       ...,
       [0.987491  , 0.012509  ],
       [0.98334663, 0.01665337],
       [0.87453299, 0.12546701]], shape=(97988, 2))
```

In [150…
```
model.classes_
```

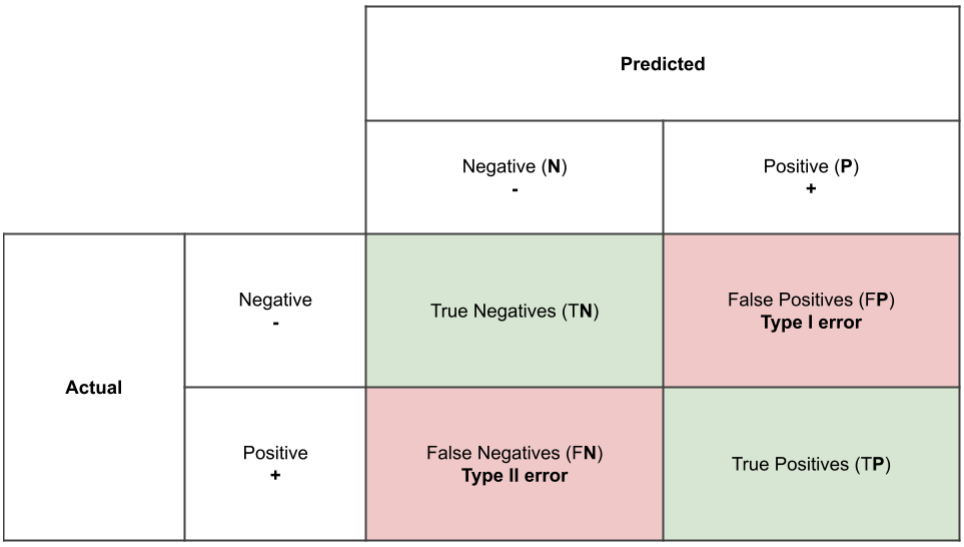
Out[150…
```
array(['No', 'Yes'], dtype=object)
```

In [151…
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
```

In [152…
```
accuracy_score(train_targets, train_preds)
```

Out[152… 0.8519206433440829

The model achieves an accuracy of 85.1% on the training set. We can visualize the breakdown of correctly and incorrectly classified inputs using a confusion matrix.



In [153…
```
confusion_matrix(train_targets, train_preds, normalize='true')
```

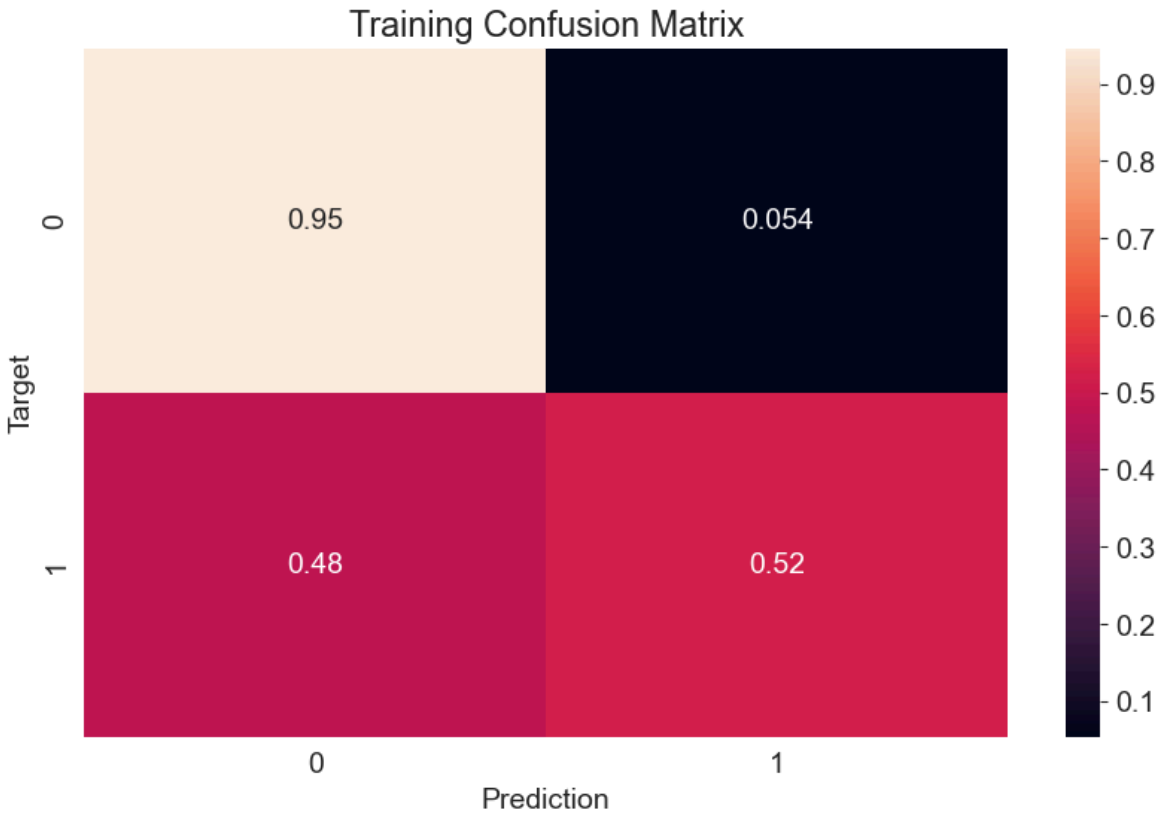
Out[153…
```
array([[0.94621341, 0.05378659],
       [0.4776585 , 0.5223415 ]])
```

Let's define a helper function to generate predictions, compute the accuracy score and plot a confusion matrix for a given st of inputs.

In [154…
```
def predict_and_plot(inputs, targets, name=''):
    preds = model.predict(inputs)

    accuracy = accuracy_score(targets, preds)
    print("Accuracy: {:.2f}%".format(accuracy * 100))

    cf = confusion_matrix(targets, preds, normalize='true')
    plt.figure()
    sns.heatmap(cf, annot=True)
    plt.xlabel('Prediction')
    plt.ylabel('Target')
    plt.title('{} Confusion Matrix'.format(name));

    return preds
```

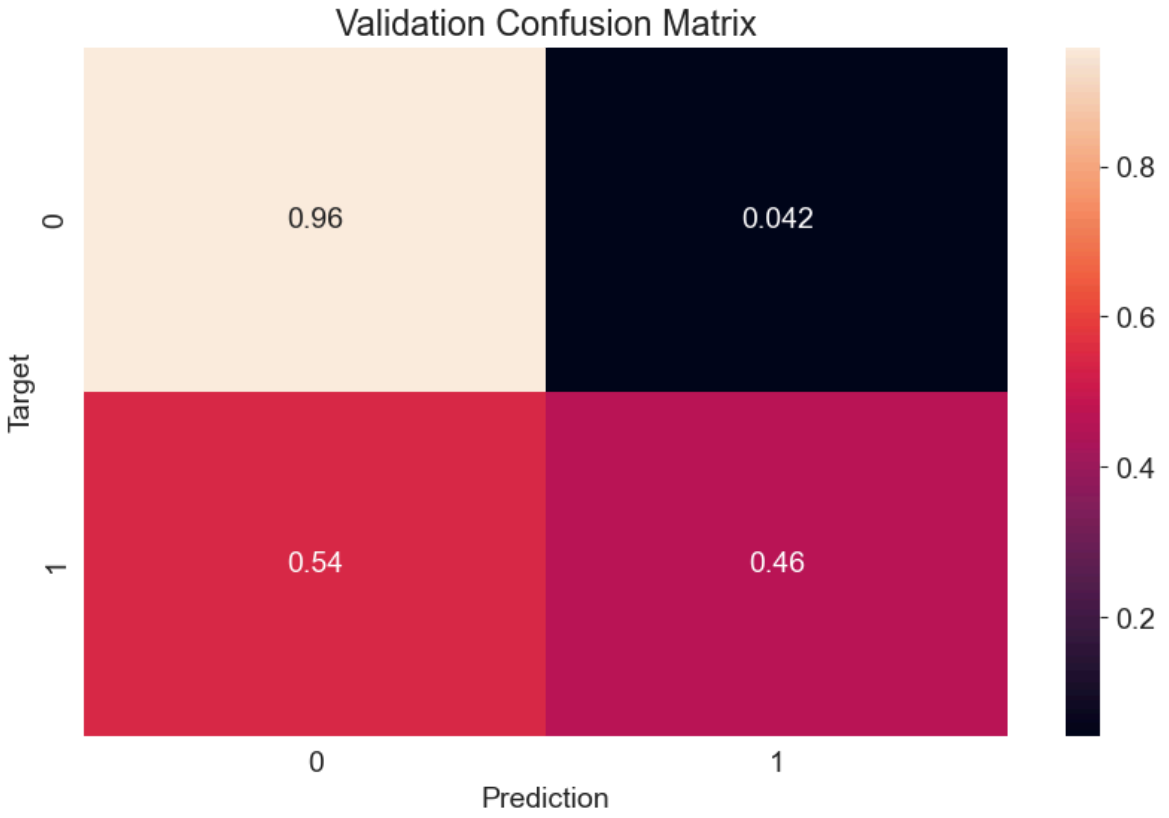
In [155…
```
train_preds = predict_and_plot(X_train, train_targets, 'Training')
```

```
Accuracy: 85.19%
```



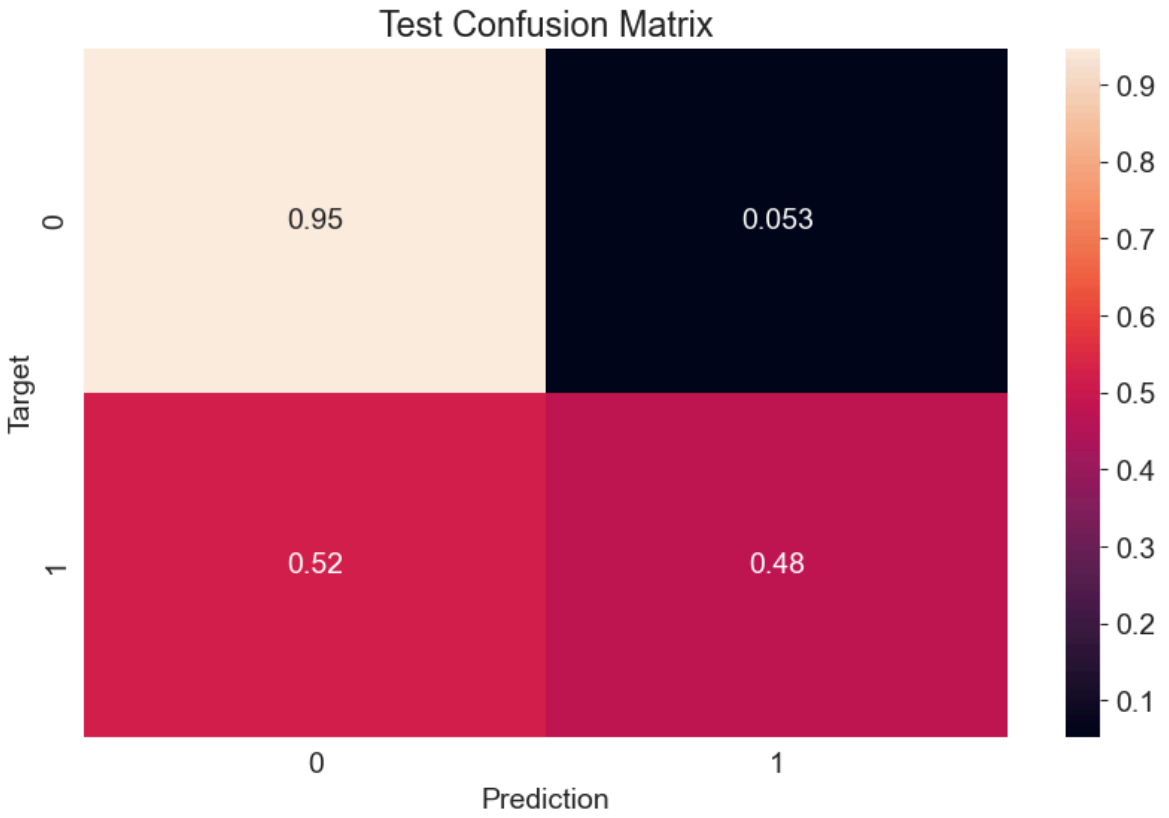
In [156…
```
val_preds = predict_and_plot(X_val, val_targets, 'Validation')
```

```
Accuracy: 85.40%
```



In [157…
```
test_preds = predict_and_plot(X_test, test_targets, 'Test')
```

```
Accuracy: 84.20%
```



In [158…
```
def random_guess(inputs):
    return np.random.choice(["No", "Yes"], len(inputs))
```

In [159…
```
def all_no(inputs):
    return np.full(len(inputs), "No")
```

In [160…
```
accuracy_score(test_targets, random_guess(X_test))
```

Out[160…
```
0.4998444185141968
```

In [161…
```
accuracy_score(test_targets, all_no(X_test))
```

Out[161…
```
0.7734344612991054
```

Our random model achieves an accuracy of 50% and our "always No" model achieves an accuracy of 77%.

Thankfully, our model is better than a "dumb" or "random" model! This is not always the case, so it's a good practice to benchmark any model you train against such baseline models.

Making Predictions on a Single Input

Once the model has been trained to a satisfactory accuracy, it can be used to make predictions on new data. Consider the following dictionary containing data collected from the Katherine weather department today.

In [162…
```
new_input = {'Date': '2021-06-19',
             'Location': 'Katherine',
```

```
            'MinTemp': 23.2,
            'MaxTemp': 33.2,
            'Rainfall': 10.2,
            'Evaporation': 4.2,
            'Sunshine': np.nan,
            'WindGustDir': 'NNW',
            'WindGustSpeed': 52.0,
            'WindDir9am': 'NW',
            'WindDir3pm': 'NNE',
            'WindSpeed9am': 13.0,
            'WindSpeed3pm': 20.0,
            'Humidity9am': 89.0,
            'Humidity3pm': 58.0,
            'Pressure9am': 1004.8,
            'Pressure3pm': 1001.5,
            'Cloud9am': 8.0,
            'Cloud3pm': 5.0,
            'Temp9am': 25.7,
            'Temp3pm': 33.0,
            'RainToday': 'Yes'}
```

In [163…
```
new_input_df = pd.DataFrame([new_input])
```

In [164…
```
new_input_df
```

Out[164…

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir
0	2021-06-19	Katherine	23.2	33.2	10.2	4.2	NaN	NNW

In [165…
```
new_input_df[numeric_cols] = imputer.transform(new_input_df[numeric_cols])
new_input_df[numeric_cols] = scaler.transform(new_input_df[numeric_cols])
new_input_df[encoded_cols] = encoder.transform(new_input_df[categorical_cols])
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
```

```
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\4090555460.py:3: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

In [166…

```
X_new_input = new_input_df[numeric_cols + encoded_cols]
X_new_input
```

Out[166…

	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustSpeed	WindSpeed9
0	0.747642	0.718336	0.027493	0.028966	0.526244	0.356589	

In [167…

```
prediction = model.predict(X_new_input)[0]
```

In [168…

```
prediction
```

Out[168…

```
'Yes'
```

In [169…

```
prob = model.predict_proba(X_new_input)[0]
```

In [170…

```
prob
```

Out[170…

```
array([0.48102595, 0.51897405])
```

In [171…

```
def predict_input(single_input):
    input_df = pd.DataFrame([single_input])
    input_df[numeric_cols] = imputer.transform(input_df[numeric_cols])
    input_df[numeric_cols] = scaler.transform(input_df[numeric_cols])
    input_df[encoded_cols] = encoder.transform(input_df[categorical_cols])
```

```
    X_input = input_df[numeric_cols + encoded_cols]
    pred = model.predict(X_input)[0]
    prob = model.predict_proba(X_input)[0][list(model.classes_).index(pred)]
    return pred, prob
```

In [172…

```
new_input = {'Date': '2021-06-19',
             'Location': 'Launceston',
             'MinTemp': 23.2,
             'MaxTemp': 33.2,
             'Rainfall': 10.2,
             'Evaporation': 4.2,
             'Sunshine': np.nan,
             'WindGustDir': 'NNW',
             'WindGustSpeed': 52.0,
             'WindDir9am': 'NW',
             'WindDir3pm': 'NNE',
             'WindSpeed9am': 13.0,
             'WindSpeed3pm': 20.0,
             'Humidity9am': 89.0,
             'Humidity3pm': 58.0,
             'Pressure9am': 1004.8,
             'Pressure3pm': 1001.5,
             'Cloud9am': 8.0,
             'Cloud3pm': 5.0,
             'Temp9am': 25.7,
             'Temp3pm': 33.0,
             'RainToday': 'Yes'}
```

In [173…

```
predict_input(new_input)
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
```

```
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

```
C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`

C:\Users\PRACHITA\AppData\Local\Temp\ipykernel_29216\2435838903.py:5: Performance
Warning:

DataFrame is highly fragmented.  This is usually the result of calling `frame.ins
ert` many times, which has poor performance.  Consider joining all columns at onc
e using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe =
frame.copy()`
```

Out[173…

```
('Yes', np.float64(0.6474964623658506))
```

In [174…

```
raw_df.Location.unique()
```

Out[174…

```
array(['Albury', 'BadgerysCreek', 'Cobar', 'CoffsHarbour', 'Moree',
       'Newcastle', 'NorahHead', 'NorfolkIsland', 'Penrith', 'Richmond',
       'Sydney', 'SydneyAirport', 'WaggaWagga', 'Williamtown',
       'Wollongong', 'Canberra', 'Tuggeranong', 'MountGinini', 'Ballarat',
       'Bendigo', 'Sale', 'MelbourneAirport', 'Melbourne', 'Mildura',
       'Nhil', 'Portland', 'Watsonia', 'Dartmoor', 'Brisbane', 'Cairns',
       'GoldCoast', 'Townsville', 'Adelaide', 'MountGambier', 'Nuriootpa',
       'Woomera', 'Albany', 'Witchcliffe', 'PearceRAAF', 'PerthAirport',
       'Perth', 'SalmonGums', 'Walpole', 'Hobart', 'Launceston',
       'AliceSprings', 'Darwin', 'Katherine', 'Uluru'], dtype=object)
```

Thank You